

板子

目录

让代码跑起来

- 编译运行
- **GDB**
- 对拍器
- 交互器

数据结构

- 并查集
- 树状数组
- 树的倍增数组
- 线段树
- 懒标记线段树
- **ST表**
- 值域线段树

图论

- **Dijkstra**
- **Floyd**
- **SPFA**
- **Kruskal**
- **Prim**
- **Boruvka**
- **Tarjan**
- **2SAT**
- 匈牙利算法
- **Dinic**
- 费用流

动态规划

- 背包**DP**
- 区间**DP**

- [数位DP](#)
- [树上DP](#)
- [换根DP](#)

数学

- [快速幂](#)
- [组合数](#)
- [Lucas定理](#)
- [质数筛](#)
- [矩阵快速幂](#)
- [EXGCD](#)
- [中国剩余定理](#)
- [高斯消元](#)
- [斯特林数](#)
- [整除分块](#)
- [错位排列](#)
- [线性基](#)
- [FFT](#)
- [NTT](#)
- [FWT](#)
- [多项式插值](#)
- [离散对数](#)

计算几何

- [点和向量](#)
- [直线](#)
- [线段](#)
- [多边形](#)
- [圆](#)
- [平面凸包](#)
- [旋转卡壳](#)
- [最近点对](#)
- [半平面交](#)
- [扫描线](#)
- [Pick定理](#)
- [圆与多边形相交](#)
- [凸多边形](#)
- [KDTree](#)
- [李超线段树](#)

字符串

- [KMP](#)
- [Manacher](#)
- 字典树
- 字符串哈希
- [AC自动机](#)
- 后缀数组
- [Height数组](#)
- 后缀自动机
- [Lyndon分解](#)
- 最小表示法
- 序列自动机
- 回文自动机
- [Z函数](#)

其他

- [最长上升子序列](#)
- [高精度整数](#)
- [快速读入](#)

让代码跑起来

编译运行

```
#!/usr/bin/env bash
g++ -std=c++20 -O2 -DLOCAL $1.cpp -o $1
./$1
rm -rf $1
```

GDB

```
#!/usr/bin/env bash
g++ -std=c++20 -Og -g $1.cpp -o $1
gdb ./1
```

```
# 打开pretty print
set print pretty on
set pagination off
set print element 0
# 运行
run
```

```

# 如果从文件读取输入
# run < input.txt
# 断点
b main # main函数
b 23 # 23行
i b # 查看断点
delete 1 # 删除1号断点
disable 1 # 禁用1号断点
enable 1 # 启用1号断点
b dfs if u == 21 && fa != -1 # 条件断点
condition 1 argc != 1 # 更改为条件断点
condition 2 # 更改为无条件
c # 在断点处暂停后，继续运行，直到走到下一个断点
n # 单步执行，不进入函数
s # 单步执行，进入函数
# 打印数据
p x # 输出x的值
p arr[0]@10 # 打印arr[0]到arr[0+10-1]
watch x # 当暂停时显示
display x # 当x更改时显示
# 调用函数
p nums.size()
call nums.push_back(2)
# 设置变量
set var x = 5 # 给x赋值为5

```

对拍器

Linux Shell

```

#!/usr/bin/env bash

g++ gen.cpp -o gen -std=c++20 -O2 || exit 1
g++ sol1.cpp -o sol1 -std=c++20 -O2 || exit 1
g++ sol2.cpp -o sol2 -std=c++20 -O2 || exit 1
mkdir -p tc
cnt=1
while [ $cnt -le 100000 ]; do
    echo "Running test $cnt"
    ./gen > tc/input.txt
    timeout 2s ./sol1 < tc/input.txt > tc/sol1.txt || exit 2
    timeout 2s ./sol2 < tc/input.txt > tc/sol2.txt || exit 2
    if ! diff -wB tc/sol1.txt tc/sol2.txt > /dev/null; then
        echo -e "Wrong Answer on test case $cnt"
        echo "Input:"
        cat tc/input.txt
    fi
    cnt=$((cnt + 1))
done

```

```

        echo -e "\nSol1:"
        cat tc/sol1.txt
        echo -e "\nSol2:"
        cat tc/sol2.txt
        exit 1
    fi
    echo -e "Accepted"
    ((cnt++))
done

```

Windows PowerShell

```

#!/usr/bin/env pwsh
Set-Alias println Write-Host
g++ gen.cpp -o gen -std=c++23 -O2
g++ sol.cpp -o sol -std=c++23 -O2
g++ sol2.cpp -o sol2 -std=c++23 -O2
mkdir -p tc -Force
$cnt = 1
while($cnt -le 100000) {
    println "Running $cnt"
    ./gen > "tc/input.txt"
    cat "tc/input.txt" | ./sol > "tc/sol.txt"
    cat "tc/input.txt" | ./sol2 > "tc/sol2.txt"
    $f1 = (cat "tc/sol.txt" -Raw) -replace '\s+', ''
    $f2 = (cat "tc/sol2.txt" -Raw) -replace '\s+', ''
    if($f1 -ne $f2) {
        println "Wrong answer" -ForegroundColor Red
        println "Input:"
        cat "tc/input.txt"
        println "Sol1:"
        cat "tc/sol.txt"
        println "Sol2:"
        cat "tc/sol2.txt"
        break
    }
    println "Accepted" -ForegroundColor Green
    $cnt++
}

```

交互器

```

#!/usr/bin/env python
from argparse import *
from subprocess import *

```

```

import sys

ap = ArgumentParser()
ap.add_argument('program', nargs='+')
ap.add_argument('-x', '--num-to-guess', type=int, dest='x', default=42)
ap.add_argument('-q', '--quiet', dest='quiet', action='store_true')
args = ap.parse_args()

def write(p: Popen[str], line):
    assert p.poll() is None and p.stdin is not None
    if not args.quiet:
        print(f'Write: {line}', flush=True)
    p.stdin.write(f"{line}\n")
    p.stdin.flush()

def read(p: Popen[str]) -> str:
    assert p.poll() is None and p.stdout is not None
    line = p.stdout.readline().strip()
    assert line != ''
    if not args.quiet:
        print(f'Read: {line}', flush=True)
    return line

def wrong(p: Popen[str], msg):
    p.kill()
    print(f"Wrong answer: {msg}")
    sys.exit(1)

q = 0
with Popen(
    " ".join(args.program), shell=True,
    stdin=PIPE, stdout=PIPE, universal_newlines=True
) as p:
    while True:
        q += 1
        if q > 50:
            wrong(p, "too many queries")
        try:
            s = read(p)
            guess = int(s)
        except:
            wrong(p, f"invalid input: {s}")
        if guess == args.x:
            write(p, '=')
            break
        elif guess > args.x:
            write(p, '>')
        else:
            write(p, '<')

```

```

sys.stdout.write("Accepted")
sys.stdout.write(f"Number of query: {q}\n")
sys.stdout.flush()

```

数据结构

并查集

```

struct DSU {
    explicit DSU(int n) : fs(n, -1) {}
    int find(int x) {
        if(fs[x] < 0) return x;
        return fs[x] = find(fs[x]);
    }
    bool is_connected(int x, int y) {
        return find(x) == find(y);
    }
    int size(int x) {
        return -fs[find(x)];
    }
    void connect(int x, int y) {
        x = find(x);
        y = find(y);
        if(x == y) return;
        int sx = size(x), sy = size(y);
        if(sx < sy) {
            fs[y] -= sx;
            fs[x] = y;
        } else {
            fs[x] -= sy;
            fs[y] = x;
        }
    }
};

private:
    vector<int> fs; // fa or size
};

```

树状数组

```

template<class T>
concept FenwickInfo = requires(T a, T b) {
    T{};

```

```

        { a + b } -> convertible_to<T>;
} && is_same_v<T, typename vector<T>::value_type>;
template<FenwickInfo T> struct Fenwick {
    explicit Fenwick(int n) : _nums(n + 1, 0), _n(n) {}
    T query(int x) const {
        T ans{};
        for(; x; x -= x & -x) ans = ans + _nums[x];
        return ans;
    }
    void update(int x, const T &v) {
        for(; x <= _n; x += x & -x) _nums[x] = _nums[x] + v;
    }

private:
    vector<T> _nums;
    int _n;
};

template<class T>
concept RangeFenwickInfo = requires(T a, T b, int c) {
    { a - b } -> convertible_to<T>;
    { a * c } -> convertible_to<T>;
    { -a } -> convertible_to<T>;
} && FenwickInfo<T>;

template<RangeFenwickInfo T> struct RangeFenwick {
    explicit RangeFenwick(int n) : _f1(n), _f2(n), _n(n) {}
    void update(int l, int r, const T &v) {
        _update(l, v);
        _update(r, -v);
    }
    T query(int x) const {
        return x * _f1.query(x) - _f2.query(x);
    }

private:
    Fenwick<T> _f1, _f2;
    int _n;
    void _update(int x, const T &v) {
        _f1.update(x, v);
        _f2.update(x, v * (x - 1));
    }
};

```

树的倍增数组

```

struct BinaryLift {
    explicit BinaryLift(int n)
        : tree(n), anc(n, vector<int>(LOG, -1)), depth(n) {}
    void addedge(int u, int v) {
        tree[u].push_back(v);
        tree[v].push_back(u);
    }
    void build(int root = 0) {
        dfs(root, -1);
    }
    int lca(int u, int v) const {
        if(depth[u] < depth[v]) swap(u, v);
        for(int k = LOG - 1; k >= 0; --k) {
            if(anc[u][k] != -1) {
                if(depth[anc[u][k]] >= depth[v]) {
                    u = anc[u][k];
                }
            }
        }
        if(u == v) return u;
        for(int k = LOG - 1; k >= 0; --k) {
            if(anc[u][k] != anc[v][k]) {
                u = anc[u][k];
                v = anc[v][k];
            }
        }
        return anc[u][0];
    }
    int kth_ancestor(int x, int k) const {
        for(int i = 0; i < LOG; ++i) {
            if((k >> i) & 1) {
                x = anc[x][i];
                if(x == -1) return -1;
            }
        }
        return x;
    }
    vector<vector<int>> tree;

private:
    static constexpr int LOG = 20;
    vector<vector<int>> anc;
    vector<int> depth;
    void dfs(int root, int fa) {
        depth[root] = fa != -1 ? depth[fa] + 1 : 0;
        anc[root][0] = fa;
    }

```

```

        for(int k = 1; k < LOG; ++k) {
            if(anc[root][k - 1] == -1) {
                anc[root][k] = -1;
            } else {
                anc[root][k] = anc[anc[root][k - 1]][k - 1];
            }
        }
        for(int v : tree[root]) {
            if(v != fa) {
                dfs(v, root);
            }
        }
    }
};

```

线段树

```

template<class Info, class Applier>
concept SegInfo = requires(Info a, Info b, const Applier src, vector<Info> vi) {
    Info{};
    Info(a);
    { a + b } -> same_as<Info>;
    { src.apply(a) } -> same_as<void>;
    { vi[0] } -> same_as<Info &>;
};

template<class Info, class Applier>
    requires(SegInfo<Info, Applier>)
struct SegTree {
    SegTree() : n(0) {}
    explicit SegTree(int sz) : n(sz), info(sz * 4, Info()) {}
    explicit SegTree(const vector<Info> &v) : n(v.size()), info(v.size() * 4) {
        _build(v, 0, 0, n);
    }
    void assign(int sz) {
        n = sz;
        info.assign(n * 4, Info());
    }
    void assign(const vector<Info> &v) {
        n = v.size();
        info.assign(n * 4, Info());
        _build(v, 0, 0, n);
    }
    Info query(int l, int r) const {
        if(l == r) return Info();
    }
};

```

```

        return _query(l, r, 0, 0, n);
    }

    void update(int x, const Applier &v) {
        _update(x, v, 0, 0, n);
    }

private:
    int n;
    vector<Info> info;
    void _build(const vector<Info> &v, int u, int rl, int rr) {
        if(rl == rr - 1) {
            info[u] = v[rl];
            return;
        }
        int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
        _build(v, ls, rl, mid);
        _build(v, rs, mid, rr);
        info[u] = info[ls] + info[rs];
    }
    Info _query(int ql, int qr, int u, int rl, int rr) const {
        if(ql <= rl && qr >= rr) return info[u];
        int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
        Info ret{};
        if(ql < mid) ret = ret + _query(ql, qr, ls, rl, mid);
        if(qr > mid) ret = ret + _query(ql, qr, rs, mid, rr);
        return ret;
    }
    void _update(int x, const Applier &v, int u, int rl, int rr) {
        if(rl == rr - 1) {
            v.apply(info[u]);
            return;
        }
        int mid = (rl + rr) >> 1, ls = u * 2 + 1, rs = u * 2 + 2;
        if(x < mid) _update(x, v, ls, rl, mid);
        else _update(x, v, rs, mid, rr);
        info[u] = info[ls] + info[rs];
    }
};

```

懒标记线段树

```

template<class Info, class Tag>
concept SegInfoTag = requires(Info a, Info b, Tag c, Tag d, int l, int r,
                                vector<Info> vi, vector<Tag> vt
                                Info{});

```

```

Tag{};
{ a + b } -> same_as<Info>;
{ c.apply(a, l, r) } -> same_as<void>;
{ c.apply(d, l, r) } -> same_as<void>;
{ c.empty() } -> same_as<bool>;
{ c.clear() } -> same_as<void>;
{ vi[l] } -> same_as<Info &>;
{ vt[l] } -> same_as<Tag &>;
};

template<class Info, class Tag>
    requires(SegInfoTag<Info, Tag>)
struct LazySegTree {
    LazySegTree() : n(0) {}
    explicit LazySegTree(int n_ : n(n_), info(4 * n_), tag(4 * n_) {})
    explicit LazySegTree(const vector<Info> &v)
        : n(v.size()), info(4 * v.size()), tag(4 * v.size()) {
        _build(v, 0, 0, n);
    }
    void assign(int n_) {
        n = n_;
        info.assign(4 * n_, Info{});
        tag.assign(4 * n_, Tag{});
    }
    void assign(const vector<Info> &v) {
        n = v.size();
        info.assign(4 * v.size(), Info{});
        tag.assign(4 * v.size(), Tag{});
        _build(v, 0, 0, n);
    }
    Info query(int l, int r) {
        return _query(l, r, 0, 0, n);
    }
    void update(int l, int r, const Tag &t) {
        _update(l, r, t, 0, 0, n);
    }

private:
    int n;
    vector<Info> info;
    vector<Tag> tag;
    void _build(const vector<Info> &v, int u, int r1, int rr) {
        if(rr - r1 == 1) {
            info[u] = v[r1];
            return;
        }
        int mid = (r1 + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
        _build(v, ls, r1, mid);

```

```

        _build(v, rs, mid, rr);
        info[u] = info[ls] + info[rs];
    }

    void _pushdown(int u, int r1, int rr) {
        if(tag[u].empty()) return;
        int mid = (r1 + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
        tag[u].apply(info[ls], r1, mid);
        tag[u].apply(info[rs], mid, rr);
        tag[u].apply(tag[ls], r1, mid);
        tag[u].apply(tag[rs], mid, rr);
        tag[u].clear();
    }

    Info _query(int ql, int qr, int u, int r1, int rr) {
        if(ql <= r1 && qr >= rr) return info[u];
        _pushdown(u, r1, rr);
        Info ret{};
        int mid = (r1 + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
        if(ql < mid) ret = ret + _query(ql, qr, ls, r1, mid);
        if(qr > mid) ret = ret + _query(ql, qr, rs, mid, rr);
        return ret;
    }

    void _update(int ul, int ur, const Tag &t, int u, int r1, int rr) {
        if(ul <= r1 && ur >= rr) {
            t.apply(info[u], r1, rr);
            t.apply(tag[u], r1, rr);
            return;
        }
        _pushdown(u, r1, rr);
        int mid = (r1 + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
        if(ul < mid) _update(ul, ur, t, ls, r1, mid);
        if(ur > mid) _update(ul, ur, t, rs, mid, rr);
        info[u] = info[ls] + info[rs];
    }
};

```

ST表

```

template<class T, class F> struct Sparse {
    Sparse(const vector<T> &v, F f) : func(f) {
        int n = v.size();
        int k = bit_width<unsigned>(n);
        table.resize(n);
        for(int i = 0; i < n; ++i) {
            table[i][0] = v[i];
        }
    }
};

```

```

        for(int j = 1; j < k; ++j) {
            int st = 1 << (j - 1);
            for(int i = 0; i + st < n; ++i) {
                table[i][j] = func(table[i][j - 1], table[i + st][j - 1]);
            }
        }
    }

    T query(int l, int r) const {
        int len = r - l;
        int j = bit_width<unsigned>(len) - 1;
        return func(table[l][j], table[r - 1 - (1 << j) + 1][j]);
    }

private:
    vector<array<T, 25>> table;
    F func;
};

template<class T, class F> Sparse(const vector<T> &v, F f) -> Sparse<T, F>;

```

值域线段树

```

// persistent segment tree
struct PST {
    explicit PST(int n_) : n(n_) {
        nodes.reserve(2 * n * (int)log2(n));
        vers.reserve(n + 1);
        vers.push_back(_build(0, n - 1));
    }

    void update(int k) {
        vers.push_back(_update(k, vers.back(), 0, n - 1));
    }

    // 这里为解决静态区间第k小问题设计，传入k, l, r
    int query(int k, int ver1, int ver2) const {
        return _query(k, vers[ver1 - 1], vers[ver2], 0, n - 1);
    }

private:
    struct Node {
        int l, r;
        int cnt;
    };

    vector<Node> nodes;
    vector<int> vers;
    int n;
    int _build(int l, int r) {

```

```

        int ret = nodes.size();
        if(l == r) {
            nodes.emplace_back(-1, -1, 0);
            return ret;
        }
        nodes.emplace_back(-1, -1, 0);
        int mid = (l + r) / 2;
        nodes[ret].l = _build(l, mid);
        nodes[ret].r = _build(mid + 1, r);
        return ret;
    }
    int _update(int k, int root, int l, int r) {
        int ret = nodes.size();
        nodes.emplace_back(nodes[root]);
        nodes[ret].cnt++;
        if(l == r) return ret;
        int mid = (l + r) / 2;
        if(k <= mid) nodes[ret].l = _update(k, nodes[root].l, l, mid);
        else nodes[ret].r = _update(k, nodes[root].r, mid + 1, r);
        return ret;
    }
    int _query(int k, int root1, int root2, int l, int r) const {
        if(l == r) return l;
        int mid = (l + r) / 2;
        int cnt1 = nodes[nodes[root2].l].cnt - nodes[nodes[root1].l].cnt;
        if(k <= cnt1) return _query(k, nodes[root1].l, nodes[root2].l, l, mid);
        else
            return _query(k - cnt1, nodes[root1].r, nodes[root2].r, mid + 1, r);
    }
};

```

图论

Dijkstra

```

template<WeightedEdgeT Edge>
vector<ll> dijkstra(const vector<vector<Edge>> &graph, int s) {
    int n = graph.size();
    vector<ll> di(n, inf);
    di[s] = 0;
    vector<bool> vis(n, false);
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
    pq.emplace(0, s);
    while(!pq.empty()) {

```

```

        auto [_, u] = pq.top();
        pq.pop();
        if(vis[u]) continue;
        vis[u] = true;
        for(auto &e : graph[u]) {
            if(!vis[e.v] && di[e.v] > di[u] + e.w) {
                di[e.v] = di[u] + e.w;
                pq.emplace(di[e.v], e.v);
            }
        }
    }
    return di;
}

```

Floyd

```

// 操作后，graph就变成了最短路
void floyd(vector<vector<ll>> &graph) {
    for(int k = 0; k < graph.size(); ++k) {
        for(int i = 0; i < graph.size(); ++i) {
            for(int j = 0; j < graph.size(); ++j) {
                chkmin(graph[i][j], graph[i][k] + graph[k][j]);
            }
        }
    }
}

```

SPFA

```

// 返回空vector说明有负环
template<WeightedEdgeT Edge>
vector<ll> spfa(const vector<vector<Edge>> &graph, int start) {
    int n = graph.size();
    vector<ll> dist(n, inf);
    vector<int> cnt(n, 0);
    vector<bool> inq(n, false);
    queue<int> q;
    q.push(start);
    dist[start] = 0;
    inq[start] = true;
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
    }
}

```

```

        for(auto &e : graph[u]) {
            if(dist[e.v] > dist[u] + e.w) {
                dist[e.v] = dist[u] + e.w;
                if(!inq[e.v]) {
                    inq[e.v] = true;
                    q.push(e.v);
                    cnt[e.v] = cnt[u] + 1;
                    if(cnt[e.v] > n) return {};
                }
            }
        }
    }
    return dist;
}

```

Kruskal

```

constexpr ll inf = 0x3f3f3f3f3f3f3f3f;

struct Edge {
    int u, v;
    ll w;
};

ll kruskal(vector<Edge> &edges, int n) {
    DSU dsu(n);
    ll ans = 0;
    sort(edges.begin(), edges.end(),
        [](const Edge &a, const Edge &b) { return a.w < b.w; });
    for(auto &e : edges) {
        if(!dsu.is_connected(e.u, e.v)) {
            dsu.connect(e.u, e.v);
            ans += e.w;
        }
    }
    for(int i = 1; i < n; ++i) {
        if(!dsu.is_connected(i, 0)) return inf;
    }
    return ans;
}

```

Prim

```

template<WeightedEdgeT Edge> ll prim(const vector<vector<Edge>> &graph) {
    int n = graph.size();
    vector<bool> vis(n, false);
    vector<ll> dist(n, inf);
    dist[0] = 0;
    priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
    pq.emplace(0, 0);
    ll ret = 0;
    while(!pq.empty()) {
        auto [w, v] = pq.top();
        pq.pop();
        if(vis[v]) continue;
        vis[v] = true;
        ret += w;
        for(auto &e : graph[v]) {
            if(!vis[e.v] && dist[e.v] > e.w) {
                dist[e.v] = e.w;
                pq.emplace(e.w, e.v);
            }
        }
    }
    for(bool b : vis) {
        if(!b) return inf;
    }
    return ret;
}

```

Boruvka

```

constexpr ll inf = 0x3f3f3f3f3f3f3f3f;

struct Edge {
    int u, v;
    ll w;
};

ll boruvka(const vector<Edge> &edges, int n) {
    int m = edges.size();
    DSU dsu(n);
    ll ans = 0;
    int c = n;
    vector<int> mst(n);
    while(c > 1) {
        for(int i = 0; i < n; ++i) {
            if(dsu.find(i) == i) mst[i] = -1;

```

```

    }
    for(int i = 0; i < m; ++i) {
        auto [u, v, w] = edges[i];
        int fu = dsu.find(u), fv = dsu.find(v);
        if(fu == fv) continue;
        if(mst[fu] == -1 || edges[mst[fu]].w > w) mst[fu] = i;
        if(mst[fv] == -1 || edges[mst[fv]].w > w) mst[fv] = i;
    }
    bool flag = false;
    for(int i = 0; i < n; ++i) {
        if(dsu.find(i) != i || mst[i] == -1) continue;
        auto [u, v, w] = edges[mst[i]];
        if(!dsu.is_connected(u, v)) {
            dsu.connect(u, v);
            ans += w;
            --c;
            flag = true;
        }
    }
    if(!flag) return inf;
}
return ans;
}

```

Tarjan

```

template<class T>
concept EdgeT = requires(T t) {
    { t.v } -> convertible_to<int>;
};

// 强连通分量
struct SCC {
    explicit SCC(int n) : nodes(n), graph(n) {}
    explicit SCC(const vector<vector<int>> &g) : graph(g), nodes(g.size()) {
        build();
    }
    template<EdgeT T>
    explicit SCC(const vector<vector<T>> &g)
        : graph(g.size()), nodes(g.size()) {
        int n = g.size();
        for(int u = 0; u < n; ++u) {
            for(const T &edge : g[u]) addedge(u, edge.v);
        }
        build();
    }

```

```

}
void addedge(int u, int v) {
    if(u != v) graph[u].push_back(v);
}
void build() {
    int dfn = 0, cnt = 0;
    for(int i = 0; i < nodes.size(); i++) {
        if(nodes[i].dfn == -1) dfs(dfn, cnt, i);
    }
    dag.assign(cnt, {});
    set<pair<int, int>> edges;
    for(int u = 0; u < graph.size(); ++u) {
        for(int v : graph[u]) {
            int bu = nodes[u].inscc, bv = nodes[v].inscc;
            if(bu != bv && !edges.contains({bu, bv})) {
                dag[bu].push_back(bv);
                edges.insert({bu, bv});
            }
        }
    }
}
struct Node {
    int dfn, low, inscc;
    bool ins;
    Node() : dfn(-1), low(-1), ins(false), inscc(-1) {}
};
vector<Node> nodes;
vector<vector<int>> graph, sccs, dag;

private:
stack<int> stk;
void dfs(int &dfn, int &cnt, int u) {
    nodes[u].dfn = nodes[u].low = dfn++;
    stk.push(u);
    nodes[u].ins = true;
    for(int v : graph[u]) {
        if(nodes[v].dfn == -1) {
            dfs(dfn, cnt, v);
            nodes[u].low = min(nodes[u].low, nodes[v].low);
        } else if(nodes[v].ins) {
            nodes[u].low = min(nodes[u].low, nodes[v].low);
        }
    }
    if(nodes[u].dfn == nodes[u].low) {
        vector<int> scc;
        int v = -1;
        while(v != u) {

```

```

        v = stk.top();
        stk.pop();
        nodes[v].ins = false;
        nodes[v].in_scc = cnt;
        scc.emplace_back(v);
    }
    sccs.emplace_back(move(scc));
    ++cnt;
}

};
// 边双
struct EBCC {
    explicit EBCC(int n) : nodes(n), graph(n), in_ebcc(n) {}
    explicit EBCC(const vector<vector<int>> &g)
        : graph(g), nodes(g.size()), in_ebcc(g.size()) {
        build();
    }
    template<EdgeT T>
    explicit EBCC(const vector<vector<T>> &g)
        : graph(g.size()), nodes(g.size()), in_ebcc(g.size()) {
        int n = g.size();
        for(int u = 0; u < n; ++u) {
            for(const T &edge : g[u]) addedge(u, edge.v);
        }
        build();
    }
    void addedge(int u, int v) {
        if(u == v) return;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    void build() {
        int dfn = 0;
        for(int i = 0; i < nodes.size(); ++i) {
            if(nodes[i].dfn == -1) dfs(i, -1, dfn);
        }
        for(int i = 0; i < ebcc.size(); ++i) {
            for(int u : ebcc[i]) in_ebcc[u] = i;
        }
    }
    vector<vector<int>> graph;
    vector<vector<int>> ebcc;
    vector<int> in_ebcc;
    struct Node {
        int dfn, low;
        bool ins;
    };
};

```

```

        Node() : dfn(-1), low(-1), ins(false) {}
};
vector<Node> nodes;

private:
stack<int> stk;
void dfs(int u, int fa, int &dfn) {
    nodes[u].dfn = nodes[u].low = dfn++;
    nodes[u].ins = true;
    stk.push(u);
    for(int v : graph[u]) {
        if(v == fa) continue;
        if(nodes[v].dfn == -1) {
            dfs(v, u, dfn);
            nodes[u].low = min(nodes[u].low, nodes[v].low);
        } else if(nodes[v].ins) {
            nodes[u].low = min(nodes[u].low, nodes[v].dfn);
        }
    }
    if(nodes[u].dfn == nodes[u].low) {
        vector<int> t;
        int n = -1;
        while(n != u) {
            n = stk.top();
            stk.pop();
            t.push_back(n);
            nodes[n].ins = false;
        }
        ebcc.emplace_back(move(t));
    }
}

};
// 点双
struct DCC {
    explicit DCC(int n) : nodes(n), graph(n) {}
    explicit DCC(const vector<vector<int>> &g) : graph(g), nodes(g.size()) {
        build();
    }
    template<EdgeT T>
    explicit DCC(const vector<vector<T>> &g)
        : graph(g.size()), nodes(g.size()) {
        int n = g.size();
        for(int u = 0; u < n; ++u) {
            for(const T &edge : g[u]) addedge(u, edge.v);
        }
        build();
    }
}

```

```

void addedge(int u, int v) {
    if(u == v) return;
    graph[u].push_back(v);
    graph[v].push_back(u);
}

void build() {
    int dfn_now = 0;
    for(int i = 0; i < nodes.size(); ++i) {
        if(nodes[i].dfn == -1) {
            dfs(i, -1, i, dfn_now);
        }
    }
}

struct Node {
    int dfn, low;
    Node() : dfn(-1), low(-1) {}
};

vector<Node> nodes;
vector<vector<int>> graph;
vector<vector<int>> dcc;

private:
stack<int> stk;
void dfs(int u, int parent, int root, int &dfn) {
    nodes[u].dfn = nodes[u].low = dfn++;
    stk.push(u);
    int child = 0;
    for(int v : graph[u]) {
        if(v == parent) continue;
        if(nodes[v].dfn == -1) {
            dfs(v, u, root, dfn);
            nodes[u].low = min(nodes[u].low, nodes[v].low);
            if(nodes[v].low >= nodes[u].dfn) {
                ++child;
                vector<int> f = {u};
                while(!stk.empty()) {
                    int x = stk.top();
                    stk.pop();
                    f.push_back(x);
                    if(x == v) break;
                }
                dcc.emplace_back(move(f));
            }
        } else {
            nodes[u].low = min(nodes[u].low, nodes[v].dfn);
        }
    }
}

```

```

        if(u == root && graph[u].empty()) {
            dcc.push_back(vector<int>{u});
            return;
        }
    }
};
// 割点
struct AP {
    explicit AP(int n) : nodes(n), graph(n) {}
    explicit AP(const vector<vector<int>> &g) : graph(g), nodes(g.size()) {
        build();
    }
    template<EdgeT T>
    explicit AP(const vector<vector<T>> &g) : graph(g.size()), nodes(g.size()) {
        int n = g.size();
        for(int u = 0; u < n; ++u) {
            for(const T &edge : g[u]) addedge(u, edge.v);
        }
        build();
    }
    void addedge(int u, int v) {
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    void build() {
        int dfn = 0;
        for(int i = 0; i < nodes.size(); ++i) {
            if(nodes[i].dfn == -1) {
                dfs(i, -1, dfn);
            }
        }
        sort(cutpoints.begin(), cutpoints.end());
        cutpoints.erase(unique(cutpoints.begin(), cutpoints.end()),
                        cutpoints.end());
    }
    struct Node {
        int dfn, low;
        Node() : dfn(-1), low(-1) {}
    };
    vector<Node> nodes;
    vector<vector<int>> graph;
    vector<int> cutpoints;

private:
    void dfs(int u, int fa, int &dfn) {
        nodes[u].dfn = nodes[u].low = dfn++;
        int child = 0;

```

```

        bool flag = false;
        for(int v : graph[u]) {
            if(nodes[v].dfn == -1) {
                ++child;
                dfs(v, u, dfn);
                nodes[u].low = min(nodes[u].low, nodes[v].low);
                if(fa != -1) {
                    flag |= (nodes[v].low >= nodes[u].dfn);
                }
            } else if(v != fa) {
                nodes[u].low = min(nodes[u].low, nodes[v].dfn);
            }
        }
        if(fa == -1) flag = (child > 1);
        if(flag) cutpoints.push_back(u);
    }
};

// 桥
struct Bridge {
    explicit Bridge(int n) : nodes(n), graph(n) {}
    explicit Bridge(const vector<vector<int>> &g) : graph(g), nodes(g.size()) {
        build();
    }
    template<EdgeT T>
    explicit Bridge(const vector<vector<T>> &g)
        : graph(g.size()), nodes(g.size()) {
        int n = g.size();
        for(int u = 0; u < n; ++u) {
            for(const T &edge : g[u]) addedge(u, edge.v);
        }
        build();
    }
    void addedge(int u, int v) {
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    void build() {
        int dfn = 0;
        for(int i = 0; i < nodes.size(); ++i) {
            if(nodes[i].dfn == -1) dfs(i, -1, dfn);
        }
    }
    struct Node {
        int dfn;
        int low;
        Node() : dfn(-1), low(-1) {}
    };
};

```

```

vector<Node> nodes;
vector<vector<int>> graph;
vector<pair<int, int>> bridges;

private:
void dfs(int u, int fa, int &dfn) {
    nodes[u].dfn = nodes[u].low = dfn++;
    for(int v : graph[u]) {
        if(nodes[v].dfn == -1) {
            dfs(v, u, dfn);
            nodes[u].low = min(nodes[u].low, nodes[v].low);
            if(nodes[v].low > nodes[u].dfn) {
                bridges.emplace_back(min(u, v), max(u, v));
            }
        } else if(v != fa) {
            nodes[u].low = min(nodes[u].low, nodes[v].dfn);
        }
    }
}

};

```

2SAT

```

// 本题给的是析取式，在这里我转化为了蕴含式求解
// 请打一个SCC下来
// 注意：蕴含式也要加另一条边，A->B要加B'->A'
vector<int> twosat(int n, const vector<tuple<int, int, int, int>> &conds) {
    SCC scc(2 * n);
    for(auto [i, fi, j, fj] : conds) {
        scc.addedge(2 * i + 1 - fi, 2 * j + fj);
        scc.addedge(2 * j + 1 - fj, 2 * i + fi);
    }
    scc.build();
    for(int i = 0; i < n; ++i) {
        if(scc.nodes[2 * i].inscc == scc.nodes[2 * i + 1].inscc) return {};
    }
    vector<int> ans(n, -1);
    for(int i = 0; i < scc.sccs.size(); ++i) {
        for(int ii : scc.sccs[i]) {
            int u = ii / 2;
            int v = ii % 2;
            if(ans[u] == -1) ans[u] = v;
        }
    }
}

```

```

        return ans;
    }

```

匈牙利算法

```

int hungarian(const vector<vector<int>> &graph, int vsz) {
    int usz = graph.size();
    vector<int> mu(usz, -1);
    vector<int> mv(vsz, -1);
    auto dfs = [&](auto &&dfs, int u, vector<bool> &vis) -> bool {
        for(int v : graph[u]) {
            if(vis[v]) continue;
            vis[v] = true;
            if(mv[v] == -1 || dfs(dfs, mv[v], vis)) {
                mv[v] = u;
                mu[u] = v;
                return true;
            }
        }
        return false;
    };
    int ret = 0;
    for(int u = 0; u < usz; ++u) {
        if(mu[u] == -1) {
            vector<bool> vis(vsz, false);
            if(dfs(dfs, u, vis)) ret++;
        }
    }
    return ret;
}

```

Dinic

```

constexpr ll inf = 0x3f3f3f3f3f3f3f3f;

struct Dinic {
    struct Edge {
        int to, rev;
        ll cap;
    };
    vector<vector<Edge>> graph;
    Dinic(int n, int s, int e) : graph(n), level(n), start(s), end(e), n(n) {}
    void addedge(int u, int v, ll w) {
        int iv = graph[v].size(), iu = graph[u].size();

```

```

        graph[u].push_back({v, iv, w});
        graph[v].push_back({u, iu, 0});
    }
    ll maxflow() {
        ll mf = 0;
        while(bfs()) {
            ll flow = dfs(start, inf);
            while(flow > 0) {
                mf += flow;
                flow = dfs(start, inf);
            }
        }
        return mf;
    }

private:
    vector<int> level;
    int n, start, end;
    bool bfs() {
        fill(level.begin(), level.end(), -1);
        level[start] = 0;
        queue<int> q;
        q.emplace(start);
        while(!q.empty()) {
            int u = q.front();
            q.pop();
            for(const auto &e : graph[u]) {
                if(level[e.to] == -1 && e.cap > 0) {
                    level[e.to] = level[u] + 1;
                    q.emplace(e.to);
                }
            }
        }
        return level[end] != -1;
    }
    ll dfs(int u, ll mf) {
        if(u == end || mf == 0) return mf;
        ll nf = 0;
        for(auto &e : graph[u]) {
            if(level[e.to] == level[u] + 1 && e.cap > 0) {
                ll min_flow = min(mf, e.cap);
                ll push = dfs(e.to, min_flow);
                if(push > 0) {
                    e.cap -= push;
                    graph[e.to][e.rev].cap += push;
                    nf += push;
                    mf -= push;
                }
            }
        }
        return nf;
    }

```

```

        if(mf == 0) return nf;
    }
}
}
return nf;
}
};

```

费用流

```

constexpr ll inf = 0x3f3f3f3f3f3f3f3f;

struct EK {
    struct Edge {
        int dst, rev;
        ll weight, flow;
        bool neg;
    };
    vector<vector<Edge>> graph;
    int n;
    explicit EK(int _n) : n(_n), graph(_n) {}
    void addedge(int u, int v, ll flow, ll weight) {
        int iu = graph[u].size(), iv = graph[v].size();
        graph[u].push_back({v, iv, weight, flow, false});
        graph[v].push_back({u, iu, -weight, 0, true});
    }
    pair<ll, ll> mcmf(int s, int t) {
        ll cost = 0, flow = 0;
        while(true) {
            vector<int> pv(n, -1), pe(n, -1);
            vector<ll> d(n, inf), mf(n, 0);
            vector<bool> inq(n, false);
            queue<int> q;
            q.push(s);
            d[s] = 0;
            mf[s] = inf;
            inq[s] = true;
            while(!q.empty()) {
                int u = q.front();
                q.pop();
                inq[u] = false;
                for(int i = 0; i < graph[u].size(); ++i) {
                    auto &e = graph[u][i];
                    int v = e.dst;
                    ll f = e.flow, w = e.weight;

```

```

        if(f > 0 && d[u] + w < d[v]) {
            d[v] = d[u] + w;
            pv[v] = u;
            pe[v] = i;
            mf[v] = min(mf[u], f);
            if(!inq[v]) {
                q.push(v);
                inq[v] = true;
            }
        }
    }

    }

    if(mf[t] <= 0) break;
    ll add = mf[t];
    for(int v = t; v != s; v = pv[v]) {
        int u = pv[v], e = pe[v];
        graph[u][e].flow -= add;
        graph[v][graph[u][e].rev].flow += add;
    }
    flow += add;
    cost += add * d[t];
}

return {cost, flow};
}

};

```

动态规划

背包DP

```

struct Item {
    ll w, v;
};

ll knapsack_01(const vector<Item> &s, int mw) {
    vector<ll> dp(mw + 1, 0);
    for(auto [w, v] : s) {
        for(int ww = mw; ww >= w; --ww) {
            chkmax(dp[ww], dp[ww - w] + v);
        }
    }
    return dp[mw];
}

ll knapsack_full(const vector<Item> &s, int mw) {

```

```

vector<ll> dp(mw + 1, 0);
for(auto [w, v] : s) {
    for(int ww = w; ww <= mw; ++ww) {
        chkmax(dp[ww], dp[ww - w] + v);
    }
}
return dp[mw];
}

struct MultiItem {
    ll w, v, cnt;
};

ll knapsack_multi(const vector<MultiItem> &s, int mw) {
    vector<Item> ss;
    for(auto [w, v, cnt] : s) {
        ll i = 1;
        while(i < cnt) {
            ss.push_back({w * i, v * i});
            cnt -= i;
            i *= 2;
        }
        ss.push_back({w * cnt, v * cnt});
    }
    return knapsack_01(ss, mw);
}

```

区间DP

```

ll range_dp(const vector<ll> &nums) {
    int n = nums.size();
    vector<ll> a(nums);
    a.insert(a.end(), nums.begin(), nums.end());
    vector<ll> sum(2 * n + 1);
    for (int i = 0; i < 2 * n; ++i) sum[i + 1] = sum[i] + a[i];
    vector<vector<ll>> dp(2 * n + 1, vector<ll>(2 * n + 1, 0));
    for (int len = 2; len <= n; ++len) {
        for (int i = 0; i + len <= 2 * n; ++i) {
            int j = i + len;
            ll best = 0;
            for (int k = i + 1; k < j; ++k) {
                best = max(best, dp[i][k] + dp[k][j]);
            }
            dp[i][j] = best + sum[j] - sum[i];
        }
    }
}

```

```

}
ll ans = 0;
for (int i = 0; i < n; ++i) ans = max(ans, dp[i][i + n]);
return ans;
}

```

数位DP

```

ll digit_dp(const string &k, int d) {
    constexpr ll modulo = 1'000'000'007;
    vector<vector<ll>> memo(k.size(), vector<ll>(d, -1));
    auto dfs = [&](auto &&dfs, int idx, int psum, bool isnum, bool islimit) -> ll {
        if (idx == (int)k.size()) return (ll)(isnum && psum == 0);
        if (isnum && !islimit && memo[idx][psum] != -1) return memo[idx][psum];
        ll ans = 0;
        if (!isnum) ans = (ans + dfs(dfs, idx + 1, 0, false, false)) % modulo;
        int llim = isnum ? 0 : 1;
        int hlim = islimit ? (k[idx] - '0') : 9;
        for (int x = llim; x <= hlim; ++x) {
            ans = (ans + dfs(dfs, idx + 1, (psum + x) % d, true, islimit && x == hlim)) % modulo;
        }
        if (isnum && !islimit) memo[idx][psum] = ans;
        return ans;
    };
    return dfs(dfs, 0, 0, false, true);
}

```

数学

快速幂

```

template<ll modulo> constexpr ll qpow(ll x, ll n) {
    ll ret = 1;
    for(; n; n >>= 1, x = x * x % modulo)
        if(n & 1) ret = ret * x % modulo;
    return ret;
}

template<ll modulo> struct PreprocessedPow {
    PreprocessedPow(ll k, ll maxn) {
        k %= modulo;
        m = ceil(sqrt(maxn + 1.5));
        powerk.assign(m, 1);
    }
};

```

```

        powerkm.assign(m, 1);
        for(int i = 1; i < m; ++i) {
            powerk[i] = (ll(powerk[i - 1]) * k) % modulo;
        }
        powerkm[1] = ll(powerk[m - 1]) * k % modulo;
        for(int i = 2; i < m; ++i) {
            powerkm[i] = ll(powerkm[i - 1]) * ll(powerkm[1]) % modulo;
        }
    }
    ll pow(ll n) const {
        ll i = n / m, j = n % m;
        return (ll(powerkm[i]) * ll(powerk[j])) % modulo;
    }
    ll operator()(ll n) const {
        return pow(n);
    }
    ll operator[](ll n) const {
        return pow(n);
    }
};

private:
    int m;
    vector<int> powerk, powerkm;
};

```

组合数

```

template<ll modulo, ll maxn = 500005> struct Comb {
    Comb() = delete;
    static ll fact(ll n) {
        ensure_init();
        return fac[n];
    }
    static ll invfact(ll n) {
        ensure_init();
        return ifac[n];
    }
    static ll perm(ll n, ll m) {
        ensure_init();
        if(m < 0 || m > n || n < 0) return 0;
        ll fn = fac[n], inm = ifac[n - m];
        return fn * inm % modulo;
    }
    static ll binom(ll n, ll m) {
        ensure_init();
    }
};

```

```

        if(m < 0 || m > n || n < 0) return 0;
        ll fn = fac[n], im = ifac[m], inm = ifac[n - m];
        return fn * im % modulo * inm % modulo;
    }

private:
    static inline int fac[maxn], ifac[maxn];
    static inline bool initd = false;
    static void ensure_init() {
        if(initd) return;
        fac[0] = 1;
        for(ll i = 1; i < maxn; ++i) {
            fac[i] = (ll)(fac[i - 1]) * i % modulo;
        }
        ifac[maxn - 1] = qpow<modulo>(fac[maxn - 1], modulo - 2);
        for(ll i = maxn - 2; i >= 0; --i) {
            ifac[i] = (ll)(ifac[i + 1]) * (i + 1) % modulo;
        }
    }
};

```

Lucas定理

```

struct Lucas {
    explicit Lucas(int m) : modulo(m), fact(m, 1), invfact(m, 1) {
        for(ll i = 1; i < modulo; ++i) {
            fact[i] = (ll)fact[i - 1] * i % modulo;
        }
        invfact[modulo - 1] = qpow(fact[modulo - 1], modulo - 2);
        for(ll i = modulo - 2; i >= 0; --i) {
            invfact[i] = ll(invfact[i + 1]) * (i + 1) % modulo;
        }
    }

    int binom(ll n, ll m) const {
        auto _binom = [&](int n, int m) -> ll {
            if(m < 0 || m > n) return 0;
            return ll(fact[n]) * ll(invfact[m]) * ll(invfact[n - m]) % modulo;
        };
        if(m == 0) return 1;
        return ll(binom(n / modulo, m / modulo)) *
            _binom(n % modulo, m % modulo) % modulo;
    }
};

private:
    int modulo;

```

```

vector<int> fact, invfact;

ll qpow(ll x, ll n) const {
    ll ret = 1;
    for(; n != 0; n >>= 1, x = x * x % modulo) {
        if(n & 1) ret = ret * x % modulo;
    }
    return ret;
}

};

```

质数筛

```

ll __qpow(ll x, ll n) {
    ll ret = 1;
    for(; n != 0; n >>= 1, x = x * x) {
        if(n & 1) ret = ret * x;
    }
    return ret;
}

template<class F>
concept eularsieve_func = requires(F &&f, int p, int k) {
    { f(p, k) } -> convertible_to<ll>;
};

struct EulerSieve {
    vector<int> primes, lpf, lpow;
    vector<ll> fv;
    explicit EulerSieve(int n) : lpf(n + 1), lpow(n + 1) {
        for(ll i = 2; i <= n; ++i) {
            if(lpf[i] == 0) {
                lpf[i] = i;
                lpow[i] = 1;
                primes.push_back(i);
            }
            for(ll p : primes) {
                ll j = i * p;
                if(j > n) break;
                lpf[j] = p;
                if(i % p == 0) {
                    lpow[j] = lpow[i] + 1;
                    break;
                } else {
                    lpow[j] = 1;
                }
            }
        }
    }
};

```

```

    }
}

template<eularsieve_func F>
EularSieve(int n, F &&f) : lpf(n + 1), lpow(n + 1), fv(n + 1) {
    fv[1] = 1;
    for(ll i = 2; i <= n; ++i) {
        if(lpf[i] == 0) {
            lpf[i] = i;
            lpow[i] = 1;
            primes.push_back(i);
            fv[i] = f(i, 1);
        }
        for(ll p : primes) {
            ll j = i * p;
            if(j > n) break;
            lpf[j] = p;
            if(i % p == 0) {
                lpow[j] = lpow[i] + 1;
                ll jp = __qpow(lpf[j], lpow[j]);
                ll rem = j / jp;
                if(rem == 1) {
                    fv[j] = f(p, lpow[j]);
                } else {
                    fv[j] = fv[rem] * fv[jp];
                }
                break;
            } else {
                fv[j] = fv[i] * fv[p];
                lpow[j] = 1;
            }
        }
    }
}

};

int eular_f(int p, int k) {
    return __qpow(p, k) - __qpow(p, k - 1);
}

int mobius_f(int p, int k) {
    return k == 0 ? 1 : k == 1 ? -1 : 0;
}

int factor_cnt_f(int p, int k) {
    return k + 1;
}

int factor_sum_f(int p, int k) {

```

```
return (__qpow(p, k + 1) - 111) / 11(p - 1);
```

```
}
```

矩阵快速幂

```
template<class T>
vector<vector<T>> matmul(const vector<vector<T>> &a,
                        const vector<vector<T>> &b) {
    int m = a.size(), n = a[0].size(), p = b[0].size();
    vector<vector<T>> ret(m, vector<T>(p, T(0)));
    for(int i = 0; i < m; ++i) {
        for(int j = 0; j < p; ++j) {
            for(int k = 0; k < n; ++k) {
                ret[i][j] = ret[i][j] + a[i][k] * b[k][j];
            }
        }
    }
    return ret;
}

template<class T> vector<vector<T>> matpow(vector<vector<T>> mat, ll n) {
    int M = mat.size();
    vector<vector<T>> ret(M, vector<T>(M, 0));
    for(int i = 0; i < M; ++i) {
        ret[i][i] = static_cast<T>(1);
    }
    for(; n != 0; n >= 1, mat = matmul(mat, mat)) {
        if(n & 111) ret = matmul(ret, mat);
    }
    return ret;
}
```

EXGCD

```
// @returns (gcd, x, y) so that gcd = ax + by
tuple<ll, ll, ll> exgcd(ll a, ll b) {
    if(b == 0) return tuple(a, 1, 0);
    auto [g, x, y] = exgcd(b, a % b);
    return tuple(g, y, x - (a / b) * y);
}
```

中国剩余定理

```

// @returns (a, b) so that answer is  $a + kb, k \in \mathbb{N}_+$ 
pair<ll, ll> crt(const vector<ll> &rem, const vector<ll> &mod) {
    int n = rem.size();
    ll modulo_ = 1, ans = 0;
    for(int i = 0; i < n; ++i) {
        modulo_ *= mod[i];
    }
    for(int i = 0; i < n; ++i) {
        ll m = modulo_ / mod[i];
        auto [_, b, __] = exgcd(m, mod[i]);
        ans = (ans + ((rem[i] * m) % modulo_ * b) % modulo_) % modulo_;
    }
    return pair((ans % modulo_ + modulo_) % modulo_, modulo_);
}

pair<ll, ll> excrt(const vector<ll> &rem, const vector<ll> &mod) {
    int n = rem.size();
    ll r1 = rem[0], m1 = mod[0];
    for(int i = 1; i < n; ++i) {
        ll r2 = rem[i], m2 = mod[i];
        auto [g, p, _] = exgcd(m1, m2);
        if((r2 - r1) % g != 0) {
            return {-1, -1};
        }
        ll v = m2 / g, x = (r2 - r1) / g;
        ll u = p % v * x % v;
        ll w = (u % v + v) % v;
        r1 += w * m1;
        m1 = lcm(m1, m2);
    }
    return {(r1 % m1 + m1) % m1, m1};
}

```

高斯消元

```

template<ll modulo>
optional<vector<vector<ll>>> gauss(const vector<vector<ll>> &a,
                                const vector<vector<ll>> &b) {
    int r = a.size(), n = a[0].size(), m = b[0].size(), row = 0;
    vector<vector<ll>> aug(r, vector<ll>(n + m));
    for(int i = 0; i < r; ++i) {
        for(int j = 0; j < m + n; ++j) {
            aug[i][j] = (j < n) ? a[i][j] : b[i][j - n];
        }
    }
}

```

```

vector<int> where(n, -1);
for(int col = 0; col < n && row < r; ++col) {
    int sel = row;
    while(sel < r && aug[sel][col] == 0) ++sel;
    if(sel == r) continue;
    swap(aug[row], aug[sel]);
    ll inv = qpow<modulo>(aug[row][col], modulo - 2);
    for(int j = col; j < n + m; ++j) {
        aug[row][j] = aug[row][j] * inv % modulo;
    }
    for(int i = 0; i < r; ++i) {
        if(i != row && aug[i][col]) {
            ll f = aug[i][col];
            for(int j = col; j < n + m; ++j) {
                aug[i][j] =
                    (aug[i][j] - f * aug[row][j] % modulo + modulo) %
                    modulo;
            }
        }
    }
    where[col] = row;
    ++row;
}
for(int i = row; i < r; ++i) {
    bool flag = true;
    for(int j = 0; j < n; ++j) {
        if(aug[i][j] != 0) {
            flag = false;
            break;
        }
    }
    if(flag) {
        for(int j = 0; j < m; ++j) {
            if(aug[i][n + j]) return nullopt;
        }
    }
}
for(int col = 0; col < n; ++col) {
    if(where[col] == -1) return nullopt;
}
vector<vector<ll>> ret(n, vector<ll>(m, 0));
for(int col = 0; col < n; ++col) {
    for(int j = 0; j < m; ++j) {
        ret[col][j] = aug[where[col]][n + j];
    }
}

```

```

        return ret;
    }
}

```

斯特林数

```

template<ll maxn, ll modulo> struct Sterling {
    Sterling() = delete;
    static int get(ll n, ll k) {
        ensure_init();
        if(n < 0 || k < 0 || k > n || n >= maxn) return 0;
        return ster[n][k];
    }

private:
    // static constexpr int maxn = 5005;
    static inline int ster[maxn][maxn];
    static void ensure_init() {
        static bool initd = false;
        if(initd) return;
        ster[0][0] = 1;
        for(ll i = 1; i < maxn; ++i) {
            ster[i][0] = 0;
            for(ll j = 1; j < i; ++j) {
                ster[i][j] =
                    (ll(ster[i - 1][j - 1]) + ll(ster[i - 1][j]) * j) %
                    modulo;
            }
            ster[i][i] = 1;
        }
        initd = true;
    }
};

```

整除分块

```

// 要计算\sum floor(n/i)
template<ll modulo> ll intdiv(ll n) {
    ll ans = 0;
    for(ll b = 1; b <= n; ) {
        ll val = n / b;
        ll r = n / val;
        ll len = r - b + 1;
        ans = (ans + (len % modulo * val % modulo)) % modulo;
        b = r + 1;
    }
}

```

```

        return ans;
    }
}

```

错位排列

```

template<ll modulo, ll maxn = 500005> struct Dislocation {
    Dislocation() = delete;
    static ll get(int i) {
        ensure_init();
        return nums[i];
    }

private:
    static inline ll nums[maxn];
    static inline bool initd = false;
    static void ensure_init() {
        if(initd) return;
        nums[0] = nums[2] = 1;
        for(ll i = 3; i < maxn; ++i) {
            nums[i] = (i - 1) * (nums[i - 1] + nums[i - 2]) % modulo;
        }
        return 0;
    }
};

```

线性基

```

struct LinearBasis_XOR {
    LinearBasis_XOR() : base(61) {}
    void insert(ll val) {
        for(int i = 60; i >= 0; --i) {
            if((val >> i) & 1) == 0) continue;
            if(base[i] == 0) {
                base[i] = val;
                return;
            }
            val ^= base[i];
        }
    }
    ll query_max() const {
        ll ans = 0;
        for(int i = 60; i >= 0; --i) {
            if((ans ^ base[i]) > ans) {
                ans ^= base[i];
            }
        }
    }
};

```

```

    }

    return ans;
}

private:
    vector<ll> base;

};

```

FFT

```

constexpr ld pi = numbers::pi_v<ld>;

using cd = complex<ld>;
vector<ld> multiply(const vector<ld> &a, const vector<ld> &b) {
    int n = 1;
    while(n < (a.size() + b.size())) {
        n <= 1;
    }
    vector<cd> fa(n), fb(n);
    for(int i = 0; i < a.size(); ++i) {
        fa[i] = a[i];
    }
    for(int i = 0; i < b.size(); ++i) {
        fb[i] = b[i];
    }
    auto fft = [](auto &&fft, vector<cd> &f, bool invert) -> void {
        int n = f.size();
        if(n == 1) return;
        vector<cd> f0(n / 2), f1(n / 2);
        for(int i = 0; i < n / 2; ++i) {
            f0[i] = f[2 * i];
            f1[i] = f[2 * i + 1];
        }
        fft(fft, f0, invert);
        fft(fft, f1, invert);
        ld theta = 2.1 * pi / n * (invert ? -1.1 : 1.1);
        cd wt = 1, w(cos(theta), sin(theta));
        for(int t = 0; t < n / 2; ++t) {
            cd u = f0[t], v = wt * f1[t];
            f[t] = u + v;
            f[t + n / 2] = u - v;
            wt *= w;
        }
    };
};

```

```

fft(fft, fa, false);
fft(fft, fb, false);
for(int i = 0; i < n; ++i) {
    fa[i] *= fb[i];
}
fft(fft, fa, true);
for(int i = 0; i < n; ++i) {
    fa[i] /= n;
}
vector<ld> ret(n);
for(int i = 0; i < n; ++i) {
    ret[i] = fa[i].real();
}
while(ret.size() > (a.size() + b.size() - 1)) {
    ret.pop_back();
}
return ret;
}

```

NTT

```

constexpr ll modulo = 998244353, g = 3;
inline ll qpow(ll x, ll n) {
    ll ret = 1;
    for(; n; n >>= 1, x = x * x % modulo)
        if(n & 1) ret = ret * x % modulo;
    return ret;
}

vector<ll> multiply(const vector<ll> &a, const vector<ll> &b) {
    int n = 1;
    while(n < a.size() + b.size()) {
        n <= 1;
    }
    vector<ll> ca(n), cb(n);
    for(int i = 0; i < a.size(); ++i) {
        ca[i] = a[i];
    }
    for(int i = 0; i < b.size(); ++i) {
        cb[i] = b[i];
    }
    auto ntt = [](auto &&ntt, vector<ll> &f, bool invert) -> void {
        int n = f.size();
        if(n == 1) return;
        vector<ll> f0(n / 2), f1(n / 2);

```

```

        for(int i = 0; i < n / 2; ++i){
            f0[i] = f[2 * i];
            f1[i] = f[2 * i + 1];
        }
        ntt(ntt, f0, invert);
        ntt(ntt, f1, invert);
        ll w = 1, wn = qpow(g, (modulo - 1) / n);
        if(invert) {
            wn = qpow(wn, modulo - 2);
        }
        for(int t = 0; t < n / 2; ++t) {
            ll u = f0[t], v = w * f1[t] % modulo;
            f[t] = (u + v) % modulo;
            f[t + n / 2] = (u - v + modulo) % modulo;
            w = w * wn % modulo;
        }
    };
    ntt(ntt, ca, false);
    ntt(ntt, cb, false);
    for(int i = 0; i < n; ++i) {
        ca[i] = ca[i] * cb[i] % modulo;
    }
    ntt(ntt, ca, true);
    for(int i = 0; i < n; ++i) {
        ca[i] = ca[i] * qpow(n, modulo - 2) % modulo;
    }
    while(ca.size() > (a.size() + b.size() - 1)) {
        ca.pop_back();
    }
    return ca;
}

```

FWT

```

// constexpr ll modulo = 998244353, inv2 = 499122177;
template<ll modulo> void fwt_or(vector<ll> &a, bool invert) {
    ll n = a.size(), type = invert ? -1 : 1;
    for(ll x = 2; x <= n; x <= 1) {
        ll k = x >> 1;
        for(ll i = 0; i < n; i += x)
            for(ll j = 0; j < k; ++j)
                a[i + j + k] = (a[i + j + k] + a[i + j] * type) % modulo;
    }
}

template<ll modulo> void fwt_and(vector<ll> &a, bool invert) {

```

```

    ll n = a.size(), type = invert ? -1 : 1;
    for(ll x = 2; x <= n; x <= 1) {
        ll k = x >> 1;
        for(ll i = 0; i < n; i += x)
            for(ll j = 0; j < k; ++j)
                a[i + j] = (a[i + j] + a[i + j + k] * type) % modulo;
    }
}

template<ll modulo> void fwt_xor(vector<ll> &a, bool invert) {
    constexpr ll inv2 = qpow<modulo>(2, modulo - 2);
    ll n = a.size(), type = invert ? inv2 : 1;
    for(ll x = 2; x <= n; x <= 1) {
        ll k = x >> 1;
        for(ll i = 0; i < n; i += x) {
            for(ll j = 0; j < k; ++j) {
                ll u = a[i + j], v = a[i + j + k];
                a[i + j] = (u + v) % modulo;
                a[i + j + k] = (u - v) % modulo;
                a[i + j] = (a[i + j] * type) % modulo;
                a[i + j + k] = (a[i + j + k] * type) % modulo;
            }
        }
    }
}

template<ll modulo> void fwt_xnor(vector<ll> &a, bool invert) {
    reverse(a.begin(), a.end());
    fwt_xor<modulo>(a, invert);
    reverse(a.begin(), a.end());
}

template<ll modulo>
vector<ll> fwt_transform(const vector<ll> &a, const vector<ll> &b,
                        void (*func)(vector<ll> &a, bool invert))

    ll n = 1;
    while(n < max(a.size(), b.size())) n <= 1;
    vector<ll> A(n, 0), B(n, 0), C(n, 0);
    for(ll i = 0; i < a.size(); i++) A[i] = a[i];
    for(ll i = 0; i < b.size(); i++) B[i] = b[i];
    func(A, false);
    func(B, false);
    for(ll i = 0; i < n; i++) {
        C[i] = (A[i] * B[i]) % modulo;
    }
    func(C, true);
    for(ll i = 0; i < n; i++) {
        C[i] = (C[i] % modulo + modulo) % modulo;
    }
}

```

```
return C;
```

```
}
```

多项式插值

```
template<ll modulo>
ll lagrange(const vector<ll> &x, const vector<ll> &y, ll x0) {
    int n = x.size();
    vector<ll> iden(n);
    for(int i = 0; i < n; ++i) {
        ll den = 1;
        for(int j = 0; j < n; ++j) {
            if(i != j) den = den * (x[i] - x[j] + modulo) % modulo;
        }
        iden[i] = qpow(den, modulo - 2);
    }
    ll ret = 0;
    for(int i = 0; i < n; ++i) {
        ll num = 1;
        for(int j = 0; j < n; ++j) {
            if(i != j) num = num * (x0 - x[j] + modulo) % modulo;
        }
        ret = (ret + y[i] * num % modulo * iden[i]) % modulo;
    }
    return ret;
}

// if x[i] = i for i = 0 to x.size() - 1, then call this to solve in O(n)
template<ll modulo> ll lagrange(const vector<ll> &y, ll x0) {
    int n = y.size();
    vector<ll> pre(n), suf(n);
    pre[0] = (x0 - 0 + modulo) % modulo;
    for(int i = 1; i < n; ++i) {
        pre[i] = pre[i - 1] * (x0 - i + modulo) % modulo;
    }
    suf[n - 1] = (x0 - (n - 1) + modulo) % modulo;
    for(int i = n - 2; i >= 0; --i) {
        suf[i] = suf[i + 1] * (x0 - i + modulo) % modulo;
    }
    ll ret = 0;
    for(int i = 0; i < n; ++i) {
        ll term = y[i];
        if(i > 0) term = term * pre[i - 1] % modulo;
        if(i < n - 1) term = term * suf[i + 1] % modulo;
        ll den = Comb<modulo>::invfact(i) * Comb<modulo>::invfact(n - 1 - i) %
```

```

        modulo;
        if((n - 1 - i) % 2) den = modulo - den;
        ret = (ret + term * den) % modulo;
    }
    return ret;
}

```

离散对数

```

// returns ret so that qpow(ret, a) = b, -1 if not exist
template<ll modulo> ll mlog(ll a, ll b) {
    if(b == 1) return 0;
    ll t = ceil(sqrt(modulo));
    ll now = b;
    unordered_map<ll, ll, MyHash> table;
    for(int i = 0; i < t; ++i) {
        table[now] = i;
        now = now * a % modulo;
    }
    ll mi = qpow(a, t);
    now = 1;
    for(int i = 1; i <= t; ++i) {
        now = now * mi % modulo;
        if(table.contains(now)) {
            return i * t - table[now];
        }
    }
    return -1;
}

```

计算几何

点和向量

```

constexpr ld eps = 1e-9l, pi = numbers::pi_v<ld>, inf = 1e12l;
int sign(ld a) {
    return (a < -eps) ? -1 : (a > eps) ? 1 : 0;
}
int cmp(ld a, ld b) {
    if(sign((a - b) / a) == 0) return 0;
    return sign(a - b);
}
auto cmpso(ld a, ld b) {

```

```
return cmp(a, b) <=> 0;
```

```
}
```

```
struct Point {  
    ld x, y;  
    Point() : x(0.01), y(0.01) {}  
    Point(ld _x, ld _y) : x(_x), y(_y) {}  
    Point(const complex<ld> &cd) : x(cd.real()), y(cd.imag()) {}  
    operator complex<ld>() const {  
        return complex<ld>(x, y);  
    }  
    Point operator+(const Point &p) const {  
        return Point(x + p.x, y + p.y);  
    }  
    Point operator-(const Point &p) const {  
        return Point(x - p.x, y - p.y);  
    }  
    Point operator*(ld z) const {  
        return Point(z * x, z * y);  
    }  
    friend Point operator*(ld z, const Point &p) {  
        return p * z;  
    }  
    Point operator/(ld z) const {  
        return Point(x / z, y / z);  
    }  
    bool operator==(const Point &p) const {  
        return cmp(x, p.x) == 0 && cmp(y, p.y) == 0;  
    }  
    auto operator<=>(const Point &p) const {  
        auto cx = cmpso(x, p.x);  
        if(cx != 0) return cx;  
        return cmpso(y, p.y);  
    }  
    ld len2() const {  
        return x * x + y * y;  
    }  
    ld len() const {  
        return sqrt(x * x + y * y);  
    }  
    // [0, 2pi)  
    ld arg() const {  
        ld ret = atan2(y, x);  
        int c = cmp(ret, 0);  
        return c == 1 ? ret : c == 0 ? 0.01 : ret + 2 * pi;  
    }  
    Point rotate(ld a) const {
```

```

        return Point(x * cos(a) - y * sin(a), x * sin(a) + y * cos(a));
    }
    ld &operator[](int i) {
        return i == 0 ? x : y;
    }
    ld operator[](int i) const {
        return i == 0 ? x : y;
    }
};

using Vector = Point;
ld dot(const Vector &x, const Vector &y) {
    return x.x * y.x + x.y * y.y;
}
ld cross(const Vector &x, const Vector &y) {
    return x.x * y.y - x.y * y.x;
}
ld cross(const Point &o, const Point &a, const Point &b) {
    return cross(a - o, b - o);
}
bool argcmp(const Point &x, const Point &y) {
    bool bx = sign(x.y) == 1 || (sign(x.y) == 0 && sign(x.x) == 1),
        by = sign(y.y) == 1 || (sign(y.y) == 0 && sign(y.x) == 1);
    if(bx != by) return bx;
    return sign(cross(x, y)) == 1;
}
ld dist(const Point &x, const Point &y) {
    return (x - y).len();
}
ld dist2(const Point &x, const Point &y) {
    return (x - y).len2();
}
int to_left(const Vector &a, const Vector &b) {
    return sign(cross(a, b));
}
int to_left(const Point &a, const Point &b, const Point &c) {
    return sign(cross(a, b, c));
}
ld angle(const Vector &a, const Vector &b) {
    ld cosa = clamp(dot(a, b) / (a.len() * b.len()), -1.01, 1.01);
    ld ret = acos(cosa);
    if(to_left(a, b) == -1) {
        ret = 2 * pi - ret;
    }
    return ret;
}

```

直线

```
struct Line {
    Point p, v;
    Line() {}
    Line(const Point &p, const Vector &v) : p(p), v(v) {}
    ld at(ld x) const {
        if(sign(v.x) == 0) return -inf;
        ld delx = x - p.x;
        ld ratio = delx / v.x;
        return (p + ratio * v).y;
    }
};

int to_left(const Line &ln, const Point &p) {
    return to_left(ln.v, p - ln.p);
}

bool parallel(const Line &l1, const Line &l2) {
    return sign(cross(l1.v, l2.v)) == 0;
}

int is_inter(const Line &l1, const Line &l2) {
    return parallel(l1, l2) ? 0 : 1;
}

optional<Point> inter(const Line &a, const Line &b) {
    if(parallel(a, b)) return nullopt;
    Vector v = a.v * (cross(b.v, a.p - b.p) / cross(a.v, b.v));
    return a.p + v;
}

ld dist(const Point &p, const Line &ln) {
    return abs(cross(ln.v, p - ln.p)) / ln.v.len();
}

ld dist(const Line &l1, const Line &l2) {
    if(!parallel(l1, l2)) return -1.01;
    return dist(l1.p, l2);
}

Point proj(const Point &p, const Line &ln) {
    Vector v = ln.v * (dot(ln.v, p - ln.p) / (dot(ln.v, ln.v)));
    return ln.p + v;
}

bool is_on(const Point &p, const Line &ln) {
    return sign(cross(ln.v, ln.p - p)) == 0;
}

Line midperp(const Point &a, const Point &b) {
    Point mid = (a + b) / 2;
    Vector to(-(b - a).y, (b - a).x);
    return Line(mid, to);
}
```

线段

```
struct Lineseg {
    Point a, b;
    Lineseg() {}
    Lineseg(const Point &a, const Point &b) : a(_a), b(_b) {}
    ld len() const {
        return (b - a).len();
    }
    ld at(ld x) const {
        if(cmp(x, min(a.x, b.x)) == -1 || cmp(x, max(a.x, b.x)) == 1)
            return -inf;
        if(cmp(a.x, b.x) == 0) {
            if(cmp(a.x, x) != 0) return -inf;
            return max(a.y, b.y);
        }
        Line l(a, b - a);
        return l.at(x);
    }
};

int is_on(const Point &p, const Lineseg &ls) {
    if(p == ls.a || p == ls.b) return 2;
    return to_left(p - ls.a, p - ls.b) == 0 &&
        sign(dot(p - ls.a, p - ls.b)) == -1;
}

int is_inter(const Line &ln, const Lineseg &ls) {
    int a = to_left(ln, ls.a), b = to_left(ln, ls.b);
    if(a == 0 || b == 0) return 2;
    return a == b ? 0 : 1;
}

optional<Point> inter(const Line &ln, const Lineseg &ls) {
    if(!is_inter(ln, ls)) return nullopt;
    return inter(ln, Line(ls.a, ls.b - ls.a));
}

int is_inter(const Lineseg &l1, const Lineseg &l2) {
    if(is_on(l1.a, l2) || is_on(l1.b, l2) || is_on(l2.a, l1) || is_on(l2.b, l1))
        return 2;
    Line ln1(l1.a, l1.b - l1.a), ln2(l2.a, l2.b - l2.a);
    return to_left(ln1, l2.a) * to_left(ln1, l2.b) == -1 &&
        to_left(ln2, l1.a) * to_left(ln2, l1.b) == -1;
}

ld dist(const Point &p, const Lineseg &ls) {
    if(is_on(p, ls) != 0) return 0.01;
    if(sign(dot(p - ls.a, ls.b - ls.a)) == -1 ||
        sign(dot(p - ls.b, ls.a - ls.b)) == -1)
        return min(dist(p, ls.a), dist(p, ls.b));
}
```

```

Line l(ls.a, ls.b - ls.a);
return dist(p, l);
}

```

多边形

```

struct Polygon {
    vector<Point> pts;
    Polygon() {}
    Polygon(const vector<Point> &p) : pts(p) {}
    Polygon(const vector<Line> &l) {
        pts.reserve(l.size());
        for(int i = 0; i < (int)l.size(); ++i) {
            pts.emplace_back(*inter(l[i], l[(i + 1) % l.size()]));
        }
    }
    ld area() const {
        ld ret = 0.01;
        for(int i = 0; i < (int)pts.size(); ++i) {
            ret += cross(pts[i], pts[(i + 1) % pts.size()]);
        }
        return ret / 2.01;
    }
    ld circ() const {
        ld ret = 0.01;
        for(int i = 0; i < (int)pts.size(); ++i) {
            ret += (pts[i] - pts[(i + 1) % pts.size()]).len();
        }
        return ret;
    }
    int size() const {
        return pts.size();
    }
};

```

圆

```

struct Circle {
    Point c;
    ld r;
    Circle() : r(0.01) {}
    Circle(const Point &_c, ld _r) : c(_c), r(_r) {}
    ld area() const {
        return pi * r * r;
    }
};

```

```

    }
    ld circ() const {
        return 2.01 * pi * r;
    }
};

bool is_inter(const Circle &c1, const Circle &c2) {
    ld dis = (c2.c - c1.c).len();
    ld r1 = c1.r, r2 = c2.r;
    if(abs(dis) <= eps) return false;
    if(dis > r1 + r2 + eps || dis < abs(r1 - r2) - eps) return false;
    return true;
}

optional<pair<Point, Point>> inter(const Circle &c1, const Circle &c2) {
    if(!is_inter(c1, c2)) return nullopt;
    ld dis = (c2.c - c1.c).len();
    ld r1 = c1.r, r2 = c2.r;
    ld cosa =
        clamp((r1 * r1 + dis * dis - r2 * r2) / (2 * r1 * dis), -1.01, 1.01);
    ld alp = acos(cosa);
    Point v = c2.c - c1.c;
    v = v / v.len() * c1.r;
    return pair{c1.c + v.rotate(alp), c1.c + v.rotate(-alp)};
}

bool is_inter(const Circle &c, const Line &l) {
    ld d = dist(c.c, l);
    return cmp(c.r, d) != -1;
}

optional<pair<Point, Point>> inter(const Circle &c, const Line &l) {
    if(!is_inter(c, l)) return nullopt;
    Point o = c.c;
    Vector v = l.v;
    Point p = l.p - o;
    ld A = v.len2(), B = 2 * dot(p, v), C = p.len2() - c.r * c.r,
        D = max(B * B - 4 * A * C, 0.01);
    ld s = sqrt(D);
    ld q = (B > 0 ? -B - s : -B + s) / 2;
    ld t1 = q / A, t2 = C / q;
    Point i1 = l.p + v * t1, i2 = l.p + v * t2;
    return pair{i1, i2};
}

```

平面凸包

```

vector<Point> andrew(vector<Point> points) {
    sort(points.begin(), points.end(), [](const Point &a, const Point &b) {

```

```

        return a.x < b.x || (a.x == b.x && a.y < b.y);
    });
    points.erase(unique(points.begin(), points.end()), points.end());
    int n = points.size();
    if(n <= 2) return points;
    vector<Point> stk;
    for(int i = 0; i < n; ++i) {
        while(stk.size() >= 2 &&
            sign(cross(stk[stk.size() - 2], stk.back(), points[i])) <= 0)
            stk.pop_back();
        stk.push_back(points[i]);
    }
    int t = stk.size();
    for(int i = n - 2; i >= 0; --i) {
        while((int)stk.size() > t &&
            sign(cross(stk[stk.size() - 2], stk.back(), points[i])) <= 0)
            stk.pop_back();
        stk.push_back(points[i]);
    }
    stk.pop_back();
    return stk;
}

```

旋转卡壳

```

ld farthest_dist(const vector<Point> &pts) {
    int n = pts.size();
    auto hull = andrew(pts);
    int m = hull.size();
    int j = 1;
    ld ans = 0;
    for(int i = 0; i < m; ++i) {
        int u = (i + 1) % m;
        while(cross(hull[i], hull[u], hull[(j + 1) % m]) >
            cross(hull[i], hull[u], hull[j]))
            j = (j + 1) % m;
        ans =
            max({ans, (hull[i] - hull[j]).len2(), (hull[u] - hull[j]).len2()});
    }
    return ans;
}

```

最近点对

```

ld nearest(vector<Point> pts) {
    if(pts.size() == 1) return 0.01;
    ld ans = 1e181;
    sort(pts.begin(), pts.end(), [](const Point &a, const Point &b) {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    });
    auto part = [&](auto &&part, int l, int r) -> void {
        if(r - l < 5) {
            for(int i = l; i < r; ++i) {
                for(int j = i + 1; j < r; ++j) {
                    ans = min(ans, dist(pts[i], pts[j]));
                }
            }
            sort(pts.begin() + l, pts.begin() + r,
                [](const Point &a, const Point &b) { return a.y < b.y; });
            return;
        }
        int mid = (l + r) / 2;
        ld midx = pts[mid].x;
        part(part, l, mid);
        part(part, mid, r);
        inplace_merge(pts.begin() + l, pts.begin() + mid, pts.begin() + r,
            [](const Point &a, const Point &b) { return a.y <
vector<Point> tmp;
        for(int i = l; i < r; ++i) {
            if(abs(pts[i].x - midx) < ans) {
                for(int j = int(tmp.size()) - 1;
                    j >= 0 && pts[i].y - tmp[j].y < ans; --j) {
                    ans = min(ans, dist(pts[i], tmp[j]));
                }
                tmp.push_back(pts[i]);
            }
        }
    };
    part(part, 0, pts.size());
    return ans;
}

```

半平面交

```

vector<Line> half_inter(vector<Line> lines) {
    lines.push_back({{-inf, 0.01}, {0.01, -1.01}});
    lines.push_back({{inf, 0.01}, {0.01, 1.01}});
    lines.push_back({{0.01, inf}, {-1.01, 0.01}});
    lines.push_back({{0.01, -inf}, {1.01, 0.01}});
}

```

```

sort(lines.begin(), lines.end(), [](const Line &a, const Line &b) {
    int c = cmp(a.v.arg(), b.v.arg());
    if(c != 0) return c == -1;
    return sign(cross(a.v, b.p - a.p)) < 0;
});
deque<Line> dq;
auto bad = [](const Line &l, const Line &b, const Line &c) {
    if(parallel(b, c)) return false;
    auto p = *inter(b, c);
    return to_left(l, p) < 0;
};
for(auto &l : lines) {
    if(!dq.empty() && cmp(l.v.arg(), dq.back().v.arg()) == 0) {
        if(to_left(l, dq.back().p + dq.back().v) < 0) dq.back() = l;
        continue;
    }
    while(dq.size() > 1 && bad(l, dq.back(), dq[dq.size() - 2]))
        dq.pop_back();
    while(dq.size() > 1 && bad(l, dq[0], dq[1])) dq.pop_front();
    dq.push_back(l);
}
while(dq.size() > 1 && bad(dq[0], dq.back(), dq[dq.size() - 2]))
    dq.pop_back();
while(dq.size() > 1 && bad(dq.back(), dq[0], dq[1])) dq.pop_front();
vector<Line> ret(dq.begin(), dq.end());
int m = ret.size();
return m < 3 ? vector<Line>() : ret;
}

```

扫描线

```

struct Rectangle {
    Point p1, p2;
};
struct Event {
    int delta, l, r;
    ld y;
};
struct SegTree_SL {
    SegTree_SL(const vector<ld> &xs) : nodes(4 * xs.size()), xs(xs) {
        int n = xs.size();
        _build(0, 0, n);
    }
    ld query() const {
        return nodes[0].len;
    }
};

```

```

}
// 注意左闭右开
void update(int l, int r, int v) {
    _update(0, l, r, v);
}

```

private:

```

struct Node {
    int l, r, cnt;
    ld len;
};
vector<Node> nodes;
vector<ld> xs;
void _pushup(int u) {
    if(nodes[u].cnt > 0) {
        nodes[u].len = xs[nodes[u].r] - xs[nodes[u].l];
    } else if(nodes[u].r - nodes[u].l == 1) {
        nodes[u].len = 0.01;
    } else {
        int lson = (u << 1) + 1, rson = (u << 1) + 2;
        nodes[u].len = nodes[lson].len + nodes[rson].len;
    }
}
void _build(int u, int l, int r) {
    nodes[u] = {l, r, 0, 0.01};
    if(r - l > 1) {
        int mid = (l + r) >> 1, lson = (u << 1) + 1, rson = (u << 1) + 2;
        _build(lson, l, mid);
        _build(rson, mid, r);
    }
}
void _update(int u, int l, int r, int v) {
    if(nodes[u].l >= r || nodes[u].r <= l) return;
    if(nodes[u].l >= l && nodes[u].r <= r) {
        nodes[u].cnt += v;
        _pushup(u);
        return;
    }
    int lson = (u << 1) + 1, rson = (u << 1) + 2;
    _update(lson, l, r, v);
    _update(rson, l, r, v);
    _pushup(u);
}

};
ld scanline(const vector<Rectangle> &rects) {
    int n = rects.size();
    vector<ld> xs(2 * n);

```

```

vector<Event> events(2 * n);
for(int i = 0; i < n; ++i) {
    xs[2 * i] = rects[i].p1.x;
    xs[2 * i + 1] = rects[i].p2.x;
}
sort(xs.begin(), xs.end());
xs.erase(unique(xs.begin(), xs.end()), xs.end());
auto getid = [&](ld x) -> int {
    return lower_bound(xs.begin(), xs.end(), x) - xs.begin();
};
for(int i = 0; i < n; ++i) {
    events[2 * i] = {1, getid(rects[i].p1.x), getid(rects[i].p2.x),
                    rects[i].p1.y};
    events[2 * i + 1] = {-1, getid(rects[i].p1.x), getid(rects[i].p2.x),
                        rects[i].p2.y};
}
sort(events.begin(), events.end(),
      [](const Event &a, const Event &b) { return cmp(a.y, b.y) == -1; });
SegTree_SL seg(xs);
ld lasty = events[0].y;
ld ans = 0.01;
for(auto &e : events) {
    ans += seg.query() * (e.y - lasty);
    seg.update(e.l, e.r, e.delta);
    lasty = e.y;
}
return ans;
}

```

Pick定理

```

struct Point {
    ll x, y;
    Point() : x(0), y(0) {}
    Point(ll _x, ll _y) : x(_x), y(_y) {}
    Point operator+(const Point &p) const {
        return Point(x + p.x, y + p.y);
    }
    Point operator-(const Point &p) const {
        return Point(x - p.x, y - p.y);
    }
    Point operator*(ll z) const {
        return Point(z * x, z * y);
    }
    friend Point operator*(ll z, const Point &p) {

```

```

        return p * z;
    }

    bool operator==(const Point &p) const {
        return x == p.x && y == p.y;
    }

    ll len2() const {
        return x * x + y * y;
    }

    ll &operator[](int i) {
        return i == 0 ? x : y;
    }

    ll operator[](int i) const {
        return i == 0 ? x : y;
    }
};

using Vector = Point;

ll dot(const Vector &x, const Vector &y) {
    return x.x * y.x + x.y * y.y;
}

ll cross(const Vector &x, const Vector &y) {
    return x.x * y.y - x.y * y.x;
}

ll cross(const Point &o, const Point &a, const Point &b) {
    return cross(a - o, b - o);
}

bool argcmp(const Point &x, const Point &y) {
    bool bx = x.y > 0 || (x.y == 0 && x.x > 0),
        by = y.y > 0 || (y.y == 0 && y.x > 0);
    if(bx != by) return bx;
    return cross(x, y) > 0;
}

ll dist2(const Point &a, const Point &b) {
    return (a - b).len2();
}

int to_left(const Vector &a, const Vector &b) {
    ll c = cross(a, b);
    return c > 0 ? 1 : c < 0 ? -1 : 0;
}

int to_left(const Point &a, const Point &b, const Point &c) {
    ll cr = cross(a, b, c);
    return cr > 0 ? 1 : cr < 0 ? -1 : 0;
}

struct Line {
    Point p, v;
    Line() {}
    Line(const Point &p, const Vector &v) : p(_p), v(_v) {}
};

```

```

};

int to_left(const Line &ln, const Point &p) {
    return to_left(ln.v, p - ln.p);
}

bool parallel(const Line &l1, const Line &l2) {
    return cross(l1.v, l2.v) == 0;
}

int is_inter(const Line &l1, const Line &l2) {
    return parallel(l1, l2) ? 0 : 1;
}

bool is_on(const Point &p, const Line &ln) {
    return cross(ln.v, ln.p - p) == 0;
}

struct Polygon {
    vector<Point> pts;
    Polygon() {}
    Polygon(const vector<Point> &p) : pts(p) {}
    ll twice_area() const {
        ll ret = 0;
        for(int i = 0; i < pts.size(); ++i) {
            ret += cross(pts[i], pts[(i + 1) % pts.size()]);
        }
        return abs(ret);
    }
    int size() const {
        return pts.size();
    }
};

ll points_inside(const Polygon &poly) {
    ll twos = poly.twice_area();
    ll border = 0;
    for(int i = 0; i < poly.size(); ++i) {
        int j = (i + 1) % poly.size();
        Vector v = poly.pts[j] - poly.pts[i];
        border += gcd(abs(v.x), abs(v.y));
    }
    return (twos - border + 2) / 2;
}

```

圆与多边形相交

```

ld sector_area(Circle c, Point u, Point v) {
    u = u - c.c, v = v - c.c, c.c = {};

```

```

ld alp = angle(u, v);
if(cmp(alp, pi) == 1) alp -= 2 * pi;
return c.r * c.r * alp / 2;
}

ld inter_helper(Circle c, Point a, Point b) {
    a = a - c.c, b = b - c.c, c.c = {};
    ld da = a.len(), db = b.len();
    if(cmp(da, c.r) <= 0 && cmp(db, c.r) <= 0) {
        return cross(a, b) / 2;
    }
    ld di = dist(c.c, Lineseg{a, b});
    if(cmp(di, c.r) >= 0) {
        return sector_area(c, a, b);
    }
    auto [ia, ib] = *inter(c, Line{a, b - a});
    Vector d = (b - a) / (b - a).len();
    ld ta = dot(ia - a, d), tb = dot(ib - a, d);
    if(ta > tb) swap(ia, ib);
    if(cmp(c.r, da) >= 0) {
        return cross(a, ib) / 2 + sector_area(c, ib, b);
    } else if(cmp(c.r, db) >= 0) {
        return cross(ia, b) / 2 + sector_area(c, a, ia);
    } else {
        return cross(ia, ib) / 2 + sector_area(c, a, ia) +
            sector_area(c, ib, b);
    }
}

ld inter_area(const Polygon &poly, const Circle &c) {
    ld ret = 0.01;
    for(int i = 0; i < poly.size(); ++i) {
        int j = (i + 1) % poly.size();
        ret += inter_helper(c, poly.pts[i], poly.pts[j]);
    }
    return abs(ret);
}

```

凸多边形

```

struct Convex : public Polygon {
    using Polygon::Polygon;
    // todo
};

bool is_in(const Point &pt, const Convex &convex) {
    if(convex.size() == 1) {
        return pt == convex.pts[0];
    }
}

```

```

}
if(convex.size() == 2) {
    return is_on(pt, Lineseg(convex.pts[0], convex.pts[1]));
}
auto check = [](const Point &a, const Point &b, const Point &c,
                const Point &p) {
    ld c1 = cross(b - a, p - a), c2 = cross(c - b, p - b),
    c3 = cross(a - c, p - c);
    return (sign(c1) != -1 && sign(c2) != -1 && sign(c3) != -1) ||
           (sign(c1) != 1 && sign(c2) != 1 && sign(c3) != 1);
};
int n = convex.size();
Point pivot = convex.pts[0];
if((sign(cross(convex.pts[1] - pivot, pt - pivot)) == -1) ||
   (sign(cross(convex.pts[n - 1] - pivot, pt - pivot)) == 1)) {
    return false;
}
int l = 1, r = n - 1;
while(l + 1 < r) {
    int mid = (l + r) >> 1;
    if(sign(cross(convex.pts[mid] - pivot, pt - pivot)) != -1) {
        l = mid;
    } else {
        r = mid;
    }
}
int nxt = (l == n - 1) ? 1 : l + 1;
return check(pivot, convex.pts[l], convex.pts[nxt], pt);
}

optional<pair<Point, Point>> tangent(const Point &pt, const Convex &convex) {
    if(is_in(pt, convex)) return nullopt;
    int n = convex.size();
    auto peak = [&](int l, int r, bool find_r) {
        while(l < r - 1) {
            int mid = (l + r) / 2;
            if(find_r ^ (to_left(convex.pts[(mid + n - 1) % n] - pt,
                                convex.pts[mid] - pt) > 0))
                r = mid;
            else l = mid;
        }
        return l;
    };
    if(to_left(convex.pts[0] - pt, convex.pts[1] - pt) == 0) {
        int idx = peak(
            2, n, cmp(dist(convex.pts[0], pt), dist(convex.pts[1], pt)) == -1);
        return pair{convex.pts[0], convex.pts[idx]};
    }
}

```

```

bool all_left = true, all_right = true;
auto chk = [&](int x) {
    if(x == 1) all_right = false;
    if(x == -1) all_left = false;
};
chk(to_left(convex.pts[0] - pt, convex.pts[1] - pt));
chk(to_left(convex.pts[0] - pt, convex.pts[n - 1] - pt));
if(all_left || all_right) {
    int idx = peak(1, n, all_left);
    return pair{convex.pts[0], convex.pts[idx]};
}
int l = 1, r = n;
while(l < r - 1) {
    int mid = (l + r) / 2;
    if(to_left(convex.pts[0] - pt, convex.pts[mid] - pt) == 1) {
        l = mid;
    } else {
        r = mid;
    }
}
int split = l;
bool flag = (to_left(convex.pts[0] - pt, convex.pts[1] - pt) == 1);
int i1 = peak(0, split + 1, flag), i2 = peak(split, n, !flag);
return pair{convex.pts[i1], convex.pts[i2]};
}

pair<Point, Point> tangent(const Line &ln, const Convex &convex) {
    int n = convex.size();
    Vector dir(-ln.v.y, ln.v.x);
    auto find = [&](bool flag) {
        int l = 0, r = n - 1;
        while(r - l > 2) {
            int m1 = l + (r - l) / 3, m2 = r - (r - l) / 3;
            ld dot1 = dot(convex.pts[m1], dir), dot2 = dot(convex.pts[m2], dir);
            if(flag ^ (cmp(dot1, dot2) == -1)) {
                r = m2;
            } else {
                l = m1;
            }
        }
        int idx = l;
        ld ret = dot(convex.pts[l], dir);
        for(int i = l; i <= r; ++i) {
            ld now = dot(convex.pts[i], dir);
            if(flag ^ (cmp(now, ret) == -1)) {
                idx = i;
                ret = now;
            }
        }
    };
    return pair{convex.pts[idx], convex.pts[idx]};
}

```

```

    }
    return idx;
};
int i1 = find(true), i2 = find(false);
return {convex.pts[i1], convex.pts[i2]};
}

```

KDTree

```

struct TwoDTree {
    explicit TwoDTree(const vector<Point> &pts) : nodes(pts.size()) {
        int n = nodes.size();
        for(int i = 0; i < n; ++i) {
            nodes[i].pt = pts[i];
            nodes[i].ls = nodes[i].rs = -1;
        }
        auto pushup = [&](int p) {
            for(int i = 0; i < 2; ++i) {
                nodes[p].L[i] = nodes[p].U[i] = nodes[p].pt[i];
                if(nodes[p].ls != -1) {
                    chkmin(nodes[p].L[i], nodes[nodes[p].ls].L[i]);
                    chkmax(nodes[p].U[i], nodes[nodes[p].ls].U[i]);
                }
                if(nodes[p].rs != -1) {
                    chkmin(nodes[p].L[i], nodes[nodes[p].rs].L[i]);
                    chkmax(nodes[p].U[i], nodes[nodes[p].rs].U[i]);
                }
            }
        };
        auto build = [&](auto &&build, int l, int r, int k) -> int {
            if(l >= r) return -1;
            int mid = (l + r) / 2;
            nth_element(nodes.begin() + l, nodes.begin() + mid,
                        nodes.begin() + r, [&](const Node &a, const Node &b) {
                            return a.pt[k] < b.pt[k];
                        });
            nodes[mid].ls = build(build, l, mid, 1 - k);
            nodes[mid].rs = build(build, mid + 1, r, 1 - k);
            pushup(mid);
            return mid;
        };
        root = build(build, 0, n, 0);
    }
    template<class Comp> ld kth_cmp(int k) {
        k *= 2;
    }
}

```

```

        priority_queue<ld, vector<ld>, Comp> pq;
        for(int i = 0; i < nodes.size(); ++i) {
            kth(root, nodes[i].pt, pq, k, 0);
        }
        return sqrt(pq.top());
    }
    ld kth_nearest(int k) {
        return kth_cmp<less<>>(k);
    }
    ld kth_farthest(int k) {
        return kth_cmp<greater<>>(k);
    }
    template<class Comp> ld kth_cmp(const Point &p, int k) {
        k *= 2;
        priority_queue<ld, vector<ld>, Comp> pq;
        kth(root, p, pq, k, 0);
        return sqrt(pq.top());
    }
    ld kth_nearest(const Point &p, int k) {
        return kth_cmp<less<>>(p, k);
    }
    ld kth_farthest(const Point &p, int k) {
        return kth_cmp<greater<>>(p, k);
    }

private:
    struct Node {
        int ls, rs;
        Point pt;
        ld L[2], U[2];
    };
    vector<Node> nodes;
    int root;
    template<class Comp> ld cmpdist(const Point &p, const Node &node);
    template<class Comp>
    void kth(int p, const Point &q, priority_queue<ld, vector<ld>, Comp> &pq,
            int k, int dep) {
        static Comp comp;
        if(p == -1) return;
        ld dist = (nodes[p].pt - q).len2();
        if(dist > 1e-12) {
            if(pq.size() < k) {
                pq.push(dist);
            } else if(comp(dist, pq.top())) {
                pq.pop();
                pq.push(dist);
            }
        }
    }

```

```

    }
    int dim = dep % 2;
    int fi, se;
    if(q[dim] < nodes[p].pt[dim]) {
        fi = nodes[p].rs;
        se = nodes[p].ls;
    } else {
        fi = nodes[p].ls;
        se = nodes[p].rs;
    }
    if(fi != -1) {
        kth(fi, q, pq, k, dep + 1);
    }
    if(se != -1) {
        ld md = cmpdist<Comp>(q, nodes[se]);
        if(comp(md, pq.size() < k ? inf : pq.top()) || pq.size() < k) {
            kth(se, q, pq, k, dep + 1);
        }
    }
}

};

template<class Comp>
ld TwoDTree::cmpdist(const Point &p, const Node &node) = delete;
template<> ld TwoDTree::cmpdist<less<>>(const Point &p, const Node &node) {
    ld d = 0;
    for(int i = 0; i < 2; ++i) {
        if(p[i] < node.L[i]) {
            ld t = node.L[i] - p[i];
            d += t * t;
        } else if(p[i] > node.U[i]) {
            ld t = p[i] - node.U[i];
            d += t * t;
        }
    }
    return d;
}

template<> ld TwoDTree::cmpdist<greater<>>(const Point &p, const Node &node) {
    ld dx = max(abs(p.x - node.L[0]), abs(p.x - node.U[0]));
    ld dy = max(abs(p.y - node.L[1]), abs(p.y - node.U[1]));
    return dx * dx + dy * dy;
}

```

李超线段树

```

struct LiChao {
    vector<Lineseg> lines;
    explicit LiChao(int ma) : id(ma * 4, -1), n(ma) {}
    void addline(int x1, int x2, int y1, int y2) {
        Lineseg ls({ld(x1), ld(y1)}, {ld(x2), ld(y2)});
        int i = lines.size();
        lines.push_back(ls);
        _update(i, min(x1, x2), max(x1, x2), 0, 0, n - 1);
    }
    int query(int k) const {
        return _query(k, 0, 0, n - 1).second;
    }

private:
    vector<int> id;
    int n;
    static pair<ld, int> _max(const pair<ld, int> &a, const pair<ld, int> &b) {
        int cmp1 = cmp(a.first, b.first);
        if(cmp1 != 0) return (cmp1 == 1) ? a : b;
        return a.second < b.second ? a : b;
    }
    void _update(int i, int ul, int ur, int u, int rl, int rr) {
        int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
        if(ul <= rl && ur >= rr) {
            if(id[u] == -1) {
                id[u] = i;
                return;
            }
            if(cmp(lines[i].at(mid), lines[id[u]].at(mid)) == 1) swap(i, id[u]);
            if(cmp(lines[i].at(rl), lines[id[u]].at(rl)) == 1)
                _update(i, ul, ur, ls, rl, mid);
            if(cmp(lines[i].at(rr), lines[id[u]].at(rr)) == 1)
                _update(i, ul, ur, rs, mid + 1, rr);
        } else {
            if(ul <= mid) _update(i, ul, ur, ls, rl, mid);
            if(mid < ur) _update(i, ul, ur, rs, mid + 1, rr);
        }
    }
    pair<ld, int> _query(int k, int u, int rl, int rr) const {
        pair<ld, int> ret(-inf, -1);
        if(id[u] != -1) ret = {lines[id[u]].at(k), id[u]};
        if(rl == rr) return ret;
        int mid = (rl + rr) / 2, ls = u * 2 + 1, rs = u * 2 + 2;
        if(k <= mid) ret = _max(ret, _query(k, ls, rl, mid));
        else ret = _max(ret, _query(k, rs, mid + 1, rr));
        return ret;
    }
};

```

```
    }  
};
```

字符串

KMP

```
// 得到用于KMP的数组  
vector<int> kmp_f(const string &s) {  
    int n = s.size();  
    vector<int> f(n, 0);  
    for(int i = 1, j = 0; i < n; ++i) {  
        while(j > 0 && s[i] != s[j]) j = f[j - 1];  
        if(s[i] == s[j]) ++j;  
        f[i] = j;  
    }  
    return f;  
}  
  
// 寻找第一次在off后面haystack中的needle  
int kmp_find(const string &haystack, const string &needle, int off = 0) {  
    int n = needle.size(), m = haystack.size();  
    vector<int> f = kmp_f(needle);  
    for(int i = 0, j = off; j < m; ) {  
        if(haystack[j] == needle[i]) {  
            ++i;  
            ++j;  
            if(i == n) return j - n;  
        } else {  
            (i == 0) ? (++j) : (i = f[i - 1]);  
        }  
    }  
    return -1;  
}  
  
// 返回haystack中有多少个needle, 可重 (即10101有两个101)  
int kmp_count(const string &haystack, const string &needle) {  
    int n = needle.size(), m = haystack.size();  
    int ret = 0;  
    vector<int> f = kmp_f(needle);  
    for(int i = 0, j = 0; j < m; ) {  
        if(haystack[j] == needle[i]) {  
            ++i;  
            ++j;  
            if(i == n) {  
                ++ret;  
                i = 0;  
            }  
        } else {  
            (i == 0) ? (++j) : (i = f[i - 1]);  
        }  
    }  
    return ret;  
}
```

```

        i = f[i - 1];
    }
    } else {
        (i == 0) ? (++j) : (i = f[i - 1]);
    }
}
return ret;
}

```

Manacher

```

vector<int> manacher(const string &_s) {
    string s = "$";
    for(char ch : _s) {
        s += ch;
        s += '$';
    }
    int n = s.size();
    int mr = 0;
    int c = 0;
    vector<int> dp(n, 0);
    auto expand = [&](int l, int r) {
        while(l >= 0 && r < n && s[l] == s[r]) {
            --l;
            ++r;
        }
        return (r - l) / 2;
    };
    for(int i = 0; i < n; ++i) {
        int m = 2 * c - i;
        if(i >= mr) {
            dp[i] = expand(i, i);
            mr = i + dp[i];
            c = i;
        } else if(dp[m] == mr - i) {
            dp[i] = expand(i - dp[m], i + dp[m]);
            mr = i + dp[i];
            c = i;
        } else {
            dp[i] = min(dp[m], mr - i);
        }
    }
    return dp;
}

```

字典树

```
struct Trie {
    Trie() : nodes(1) {}
    void insert(const string &str) {
        int rt = 0;
        for(int i = 0; i < str.size(); ++i) {
            if((nodes[rt].nxt[getnum(str[i])]) == -1) {
                nodes[rt].nxt[getnum(str[i])] = nodes.size();
                nodes.emplace_back();
            }
            nodes[rt].cnt++;
            rt = nodes[rt].nxt[getnum(str[i])];
        }
        nodes[rt].cnt++;
        nodes[rt].end = true;
    }
    bool find(const string &str) const {
        int rt = 0;
        for(int i = 0; i < str.size(); ++i) {
            if((nodes[rt].nxt[getnum(str[i])]) == -1) {
                return false;
            }
            rt = nodes[rt].nxt[getnum(str[i])];
        }
        return nodes[rt].end;
    }
    int prefix_count(const string &str) const {
        int rt = 0;
        for(int i = 0; i < str.size(); ++i) {
            if((nodes[rt].nxt[getnum(str[i])]) == -1) {
                return 0;
            }
            rt = nodes[rt].nxt[getnum(str[i])];
        }
        return nodes[rt].cnt;
    }
}

private:
    struct Node {
        array<int, 65> nxt;
        bool end;
        int cnt;
        Node() : end(false), cnt(0) {
            nxt.fill(-1);
        }
    }
```

```

};
vector<Node> nodes;
static int getnum(char x) {
    if(x >= 'A' && x <= 'Z') return x - 'A';
    else if(x >= 'a' && x <= 'z') return x - 'a' + 26;
    else return x - '0' + 52;
}
};

```

字符串哈希

```

struct StringHash {
    explicit StringHash(const string &s)
        : p1(s.size() + 1, 0), p2(s.size() + 1, 0) {
        for(int i = 0; i < (int)s.size(); ++i) {
            p1[i + 1] = (p1[i] * _s.mul1 + s[i]) % modulo;
            p2[i + 1] = p2[i] * _s.mul2 + s[i];
        }
    }
    ull substr(int l, int r) const {
        ull ret1 = (p1[r] - p1[l] * _s.pmul1[r - l] % modulo + modulo) % modulo;
        ull ret2 = p2[r] - p2[l] * _s.pmul2[r - l];
        return (ret1 << 3) ^ (ret1 >> 5) ^ ret2;
    }

private:
    vector<ull> p1, p2;
    static constexpr ull modulo = 998244353;
    static constexpr int maxn = 500005;
    struct _Statics {
        ull mul1, mul2;
        ull pmul1[maxn], pmul2[maxn];
        _Statics() {
            ull c = (ull)chrono::steady_clock::now().time_since_epoch().count();
            mul1 = c % 131 + 131;
            mul2 = c % 13331 + 13331;
            pmul1[0] = pmul2[0] = 1;
            for(int i = 1; i < maxn; ++i) {
                pmul1[i] = pmul1[i - 1] * mul1 % modulo;
                pmul2[i] = pmul2[i - 1] * mul2;
            }
        }
    };
};
static inline _Statics _s;
};

```

AC自动机

```
struct ACAM {
    ACAM() : nodes(1) {}
    void insert(const string &str) {
        int rt = 0;
        for(int i = 0; i < (int)str.size(); ++i) {
            int id = str[i] - '0';
            if(nodes[rt].nxt[id] == -1) {
                nodes[rt].nxt[id] = nodes.size();
                nodes.emplace_back();
            }
            rt = nodes[rt].nxt[id];
        }
    }
    void build() {
        queue<int> q;
        nodes[0].fail = 0;
        for(int i = 0; i < 2; ++i) {
            int v = nodes[0].nxt[i];
            if(v != -1) {
                nodes[v].fail = 0;
                q.push(v);
            } else {
                nodes[0].nxt[i] = 0;
            }
        }
        while(!q.empty()) {
            int rt = q.front();
            q.pop();
            for(int i = 0; i < 2; ++i) {
                int v = nodes[rt].nxt[i];
                if(v != -1) {
                    nodes[v].fail = nodes[nodes[rt].fail].nxt[i];
                    q.push(v);
                } else {
                    nodes[rt].nxt[i] = nodes[nodes[rt].fail].nxt[i];
                }
            }
        }
    }
}

private:
    struct Node {
        array<int, 2> nxt;
        int fail;
    };
};
```

```

        Node() : fail(0) {
            nxt.fill(-1);
        }
    };
    vector<Node> nodes;
};

```

后缀数组

```

vector<int> suffix_array(const string &str) {
    int n = str.size(), m = 128, p = 0;
    vector<int> rk(n * 3, -1), sa(n), id(n), cnt(max(n, m), 0);
    for(int i = 0; i < n; ++i) cnt[rk[i] = str[i]]++;
    for(int i = 1; i < m; ++i) cnt[i] += cnt[i - 1];
    for(int i = n - 1; i >= 0; --i) sa[--cnt[rk[i]]] = i;
    for(int w = 1;; w <= 1, m = p + 1) {
        int cur = 0;
        for(int i = n - w; i < n; ++i) id[cur++] = i;
        for(int i = 0; i < n; ++i) {
            if(sa[i] >= w) id[cur++] = sa[i] - w;
        }
        cnt.assign(max(n, m), 0);
        for(int i = 0; i < n; ++i) cnt[rk[i]]++;
        for(int i = 1; i < m; ++i) cnt[i] += cnt[i - 1];
        for(int i = n - 1; i >= 0; --i) sa[--cnt[rk[id[i]]]] = id[i];
        p = 0;
        vector<int> oldrk = rk;
        rk[sa[0]] = 0;
        for(int i = 1; i < n; ++i) {
            if(oldrk[sa[i]] != oldrk[sa[i - 1]] ||
               oldrk[sa[i] + w] != oldrk[sa[i - 1] + w])
                ++p;
            rk[sa[i]] = p;
        }
        if(p == n - 1) break;
    }
    return sa;
}

```

Height数组

```

vector<int> height(const string &str) {
    int n = str.size();
    vector<int> sa = suffix_array(str), rk(n), h(n);

```

```

for(int i = 0; i < n; ++i) rk[sa[i]] = i;
for(int i = 0, k = 0; i < n; ++i) {
    if(rk[i] == 0) continue;
    if(k > 0) --k;
    while(str[i + k] == str[sa[rk[i] - 1] + k]) ++k;
    h[rk[i]] = k;
}
return h;
}

```

后缀自动机

```

struct SAM {
    SAM() : nodes(1, Node(-1, 0)), last(0) {}
    void insert(char ch) {
        const int id = ch - 'a';
        const int cur = nodes.size();
        nodes.emplace_back(-1, nodes[last].len + 1);
        int p = last;
        while(p != -1 && nodes[p].nxt[id] == -1) {
            nodes[p].nxt[id] = cur;
            p = nodes[p].link;
        }
        if(p == -1) {
            nodes[cur].link = 0;
        } else {
            int q = nodes[p].nxt[id];
            if(nodes[p].len + 1 == nodes[q].len) {
                nodes[cur].link = q;
            } else {
                int clone = nodes.size();
                nodes.emplace_back(nodes[q].link, nodes[p].len + 1);
                nodes[clone].nxt = nodes[q].nxt;
                while(p != -1 && nodes[p].nxt[id] == q) {
                    nodes[p].nxt[id] = clone;
                    p = nodes[p].link;
                }
                nodes[q].link = nodes[cur].link = clone;
            }
        }
        last = cur;
        ends.push_back(cur);
    }
}

ll solve() const {
    int n = nodes.size();

```

```

vector<int> indeg(n, 0), topoorder;
topoorder.reserve(n);
for(int i = 0; i < n; ++i) {
    if(nodes[i].link != -1) {
        indeg[nodes[i].link]++;
    }
}
queue<int> q;
for(int i = 0; i < n; ++i) {
    if(indeg[i] == 0) {
        q.push(i);
    }
}
while(!q.empty()) {
    int u = q.front();
    q.pop();
    topoorder.push_back(u);
    if(nodes[u].link != -1) {
        indeg[nodes[u].link]--;
        if(indeg[nodes[u].link] == 0) {
            q.push(nodes[u].link);
        }
    }
}
ll ans = 0;
vector<ll> occur(n, 0);
for(int e : ends) {
    occur[e] = 1;
}
for(int i : topoorder) {
    if(nodes[i].link != -1) {
        occur[nodes[i].link] += occur[i];
    }
    if(occur[i] != 1) {
        ans = max(ans, occur[i] * nodes[i].len);
    }
}
return ans;
}

```

private:

```

struct Node {
    array<int, 26> nxt;
    int link;
    int len;
    Node(int lk, int ln) : link(lk), len(ln) {
        nxt.fill(-1);
    }
}

```

```

    }
};
vector<Node> nodes;
vector<int> ends;
int last;
};

```

Lyndon分解

```

vector<string> duval(const string &s) {
    int n = s.size();
    vector<string> ret;
    for(int i = 0; i < n; i) {
        int j = i + 1, k = i;
        while(j < n && s[k] <= s[j]) {
            (s[k] < s[j]) ? (k = i) : (k++);
            ++j;
        }
        while(i <= k) {
            ret.push_back(s.substr(i, j - k));
            i += (j - k);
        }
    }
    return ret;
}

```

最小表示法

```

string minrep(const string &str) {
    int n = str.size(), i = 0, j = 1, k = 0;
    string s = str + str;
    while(i < n && j < n) {
        while(k < n && s[i + k] == s[j + k]) ++k;
        if(k == n) break;
        if(s[i + k] > s[j + k]) i += (k + 1);
        else j += (k + 1);
        if(i == j) ++j;
        k = 0;
    }
    return s.substr(min(i, j), n);
}

```

序列自动机

```

struct SeqAM {
    explicit SeqAM(const string &s)
        : n(s.size()), nxt(s.size() + 2, [&] {
            array<int, 26> ret;
            ret.fill(s.size() + 1);
            return ret;
        })() {
        for(int i = n - 1; i >= 0; --i) {
            nxt[i] = nxt[i + 1];
            nxt[i][s[i] - 'a'] = i + 1;
        }
    }
    bool match(const string &t) const {
        int now = 0;
        for(char c : t) {
            now = nxt[now][c - 'a'];
        }
        return now != (n + 1);
    }

private:
    int n;
    vector<array<int, 26>> nxt;
};

```

回文自动机

```

struct PAM {
    PAM() : s("#"), last(1) {
        nodes.emplace_back(-1, 0);
        nodes.emplace_back(0, 0);
    }
    int insert(char ch) {
        s += ch;
        int pos = s.size() - 1;
        int p = last, id = ch - 'a';
        while(s[pos - nodes[p].len - 1] != ch) p = nodes[p].fail;
        if(nodes[p].nxt[id] == -1) {
            int cur = nodes.size();
            nodes[p].nxt[id] = cur;
            nodes.emplace_back(nodes[p].len + 2, 0);
            if(nodes[cur].len == 1) {
                nodes[cur].fail = 1;
            } else {
                int f = nodes[p].fail;

```

```

        while(s[pos - nodes[f].len - 1] != ch) f = nodes[f].fail;
        nodes[cur].fail = nodes[f].nxt[id];
    }
    nodes[cur].cnt =
        nodes[cur].len == 1 ? 1 : nodes[nodes[cur].fail].cnt + 1;
}
last = nodes[p].nxt[id];
return nodes[last].cnt;
}

private:
    struct Node {
        int len, fail, cnt;
        array<int, 26> nxt;
        Node(int l, int f) : len(l), fail(f), cnt(0) {
            nxt.fill(-1);
        }
    };
    vector<Node> nodes;
    string s;
    int last;
};

```

Z函数

```

vector<int> zfn(const string &s) {
    int n = s.size();
    vector<int> z(n);
    for(int i = 1, l = 0, r = 0; i < n; ++i) {
        if(i <= r && z[i - l] < r - i + 1) {
            z[i] = z[i - l];
        } else {
            z[i] = max(0, r - i + 1);
            while(i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        }
        if(i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

```

其他

最长上升子序列

```
vector<int> lis(const vector<int> &nums) {
    int n = nums.size();
    vector<int> dp, pos(n), pre(n, -1);
    for(int i = 0; i < n; ++i) {
        auto it = lower_bound(dp.begin(), dp.end(), nums[i]);
        int j = it - dp.begin();
        if(it == dp.end()) dp.push_back(nums[i]);
        else *it = nums[i];
        pos[j] = i;
        if(j != 0) pre[i] = pos[j - 1];
    }
    vector<int> ret;
    for(int i = pos[dp.size() - 1]; i != -1; i = pre[i]) {
        ret.push_back(nums[i]);
    }
    reverse(ret.begin(), ret.end());
    return ret;
}
```

高精度整数

```
struct HPINT {
    HPINT() : nums(1, 0), neg(false) {}
    // intentionally implicit, allowing HPINT a = 5
    HPINT(ll val) : neg(false) {
        if(val < 0) {
            neg = true;
            val = -val;
        }
        while(val) {
            int t = val % 10;
            nums.push_back(t);
            val /= 10;
        }
        if(nums.empty()) nums = vector<int>(1, 0);
    }
    // and HPINT b = "11451419198103141592653589793238462621718281828"
    HPINT(const string &s) : neg(false) {
        int off = 0, n = s.size();
        if(s[0] == '-') {
            off = 1;
            neg = true;
        }
    }
}
```

```

        for(; off < n; ++off) nums.push_back(s[off] - '0');
        reverse(nums.begin(), nums.end());
        _norm();
    }

    explicit operator string() const {
        string ret;
        if(neg) ret += '-';
        for(int i = nums.size() - 1; i >= 0; --i) ret += (nums[i] + '0');
        return ret;
    }

    HPINT &operator+=(const HPINT &r) {
        if(neg) {
            if(r.neg) {
                _add(nums, r.nums);
            } else if(_cmp(nums, r.nums) == 1) {
                _sub(nums, r.nums);
            } else {
                vector<int> rnums = r.nums;
                _sub(rnums, nums);
                nums.swap(rnums);
                neg = false;
            }
        } else if(r.neg) {
            if(_cmp(nums, r.nums) != -1) {
                _sub(nums, r.nums);
            } else {
                vector<int> rnums = r.nums;
                _sub(rnums, nums);
                nums.swap(rnums);
                neg = true;
            }
        } else {
            _add(nums, r.nums);
        }
        _norm();
        return *this;
    }

    HPINT &operator--(const HPINT &r) {
        if(neg) {
            if(r.neg) {
                if(_cmp(nums, r.nums) == 1) {
                    _sub(nums, r.nums);
                } else {
                    vector<int> rnums = r.nums;
                    _sub(rnums, nums);
                    nums.swap(rnums);
                    neg = false;
                }
            }
        }
    }

```

```

        }
        } else {
            _add(nums, r.nums);
        }
    } else if(r.neg) {
        _add(nums, r.nums);
    } else {
        int c = _cmp(nums, r.nums);
        if(c == -1) {
            vector<int> rnums = r.nums;
            _sub(rnums, nums);
            nums.swap(rnums);
            neg = true;
        } else {
            _sub(nums, r.nums);
        }
    }
    _norm();
    return *this;
}

HPINT &operator*=(const HPINT &r) {
    neg ^= r.neg;
    _mul(nums, r.nums);
    _norm();
    return *this;
}

friend ostream &operator<<(ostream &os, const HPINT &hp) {
    if(hp.neg) os << '-';
    for(int i = hp.nums.size() - 1; i >= 0; --i) os << hp.nums[i];
    return os;
}

friend istream &operator>>(istream &is, HPINT &hp) {
    char ch = is.rdbuf()->sbumpc();
    while(ch < '0' || ch > '9') {
        if(ch == '-') hp.neg = true;
        ch = is.rdbuf()->sbumpc();
    }
    string buffer;
    while(ch >= '0' && ch <= '9') {
        buffer += ch;
        ch = is.rdbuf()->sbumpc();
    }
    hp.nums.resize(buffer.size());
    int n = buffer.size();
    for(int i = 0; i < n; ++i) hp.nums[i] = buffer[n - i - 1] - '0';
    hp._norm();
    return is;
}

```

```

}
bool operator==(const HPINT &r) const {
    return neg == r.neg && _cmp(nums, r.num) == 0;
}
strong_ordering operator<=>(const HPINT &r) const {
    if(neg != r.neg) {
        if(neg) return strong_ordering::less;
        return strong_ordering::greater;
    }
    int c = _cmp(nums, r.num);
    if(neg) c *= -1;
    return c <=> 0;
}
HPINT operator+(const HPINT &r) const {
    HPINT ret = *this;
    ret += r;
    return ret;
}
HPINT operator-(const HPINT &r) const {
    HPINT ret = *this;
    ret -= r;
    return ret;
}
HPINT operator*(const HPINT &r) const {
    HPINT ret = *this;
    ret *= r;
    return ret;
}
}

```

private:

```

vector<int> num;
bool neg;
void _norm() {
    while(num.size() > 1 && num.back() == 0) num.pop_back();
    if(num.size() == 1 && num[0] == 0) neg = false;
}
static void _add(vector<int> &a, const vector<int> &b) {
    int carry = 0;
    for(int i = 0; i < b.size(); ++i) {
        if(i >= a.size()) a.push_back(0);
        a[i] += (b[i] + carry);
        carry = 0;
        if(a[i] >= 10) {
            carry = 1;
            a[i] -= 10;
        }
    }
}
}

```

```

    int i = b.size();
    while(carry != 0) {
        if(i >= a.size()) a.push_back(0);
        a[i] += carry;
        carry = 0;
        if(a[i] >= 10) {
            carry = 1;
            a[i] -= 10;
        }
        ++i;
    }
}

// we are assuming that a >= b
static void _sub(vector<int> &a, const vector<int> &b) {
    int borrow = 0;
    for(int i = 0; i < b.size(); ++i) {
        a[i] -= (borrow + b[i]);
        borrow = 0;
        if(a[i] < 0) {
            a[i] += 10;
            borrow = 1;
        }
    }
    int i = b.size();
    while(borrow && i < a.size()) {
        a[i] -= borrow;
        borrow = 0;
        if(a[i] < 0) {
            a[i] += 10;
            borrow = 1;
        }
        ++i;
    }
    while(a.size() > 1 && a.back() == 0) a.pop_back();
}

static void _mul(vector<int> &a, const vector<int> &b) {
    using cd = complex<double>;
    constexpr double pi = 3.14159265358979323846264338327950288;
    int n = 1;
    while(n < (int)(a.size() + b.size())) n <<= 1;
    vector<cd> fa(n), fb(n);
    for(int i = 0; i < (int)a.size(); ++i) fa[i] = a[i];
    for(int i = 0; i < (int)b.size(); ++i) fb[i] = b[i];
    auto fft = [](auto &&fft, vector<cd> &f, bool invert) -> void {
        int n = f.size();
        if(n == 1) return;
        vector<cd> f0(n / 2), f1(n / 2);

```

```

        for(int i = 0; i < n / 2; ++i) {
            f0[i] = f[2 * i];
            f1[i] = f[2 * i + 1];
        }
        fft(fft, f0, invert);
        fft(fft, f1, invert);
        double theta = 2 * pi / n * (invert ? -1 : 1);
        cd wt = 1, w(cos(theta), sin(theta));
        for(int t = 0; t < n / 2; ++t) {
            cd u = f0[t], v = wt * f1[t];
            f[t] = u + v;
            f[t + n / 2] = u - v;
            wt *= w;
        }
    };

    fft(fft, fa, false);
    fft(fft, fb, false);
    for(int i = 0; i < n; ++i) fa[i] *= fb[i];
    fft(fft, fa, true);
    a.resize(n);
    int carry = 0;
    for(int i = 0; i < n; ++i) {
        int num = int(fa[i].real() / n + 0.5) + carry;
        carry = num / 10;
        a[i] = num % 10;
    }
    while(carry) {
        a.push_back(carry % 10);
        carry /= 10;
    }
    while(a.size() > 1 && a.back() == 0) a.pop_back();
}

// 1 gt, -1 lt, 0 eq
static int _cmp(const vector<int> &a, const vector<int> &b) {
    if(a.size() > b.size()) return 1;
    if(a.size() < b.size()) return -1;
    for(int n = a.size(), i = n - 1; i >= 0; --i) {
        if(a[i] > b[i]) return 1;
        if(a[i] < b[i]) return -1;
    }
    return 0;
}

};

```

```

struct Qread {
    Qread() : state(true) {
        cin.rdbuf()->pubsetbuf(buffer, maxn);
    }
    template<integral T> Qread &operator>>(T &val) {
        if(!state) {
            val = 0;
            return *this;
        }
        T x = 0, f = 1;
        char ch = cin.rdbuf()->sbumpc();
        while(ch < '0' || ch > '9') {
            if(ch == char_traits<char>::eof()) {
                state = false;
                cin.setstate(ios_base::eofbit);
                val = x * f;
                return *this;
            }
            if(ch == '-') {
                f = -1;
            }
            ch = cin.rdbuf()->sbumpc();
        }
        while(ch >= '0' && ch <= '9') {
            x = x * 10 + ch - '0';
            ch = cin.rdbuf()->sbumpc();
        }
        val = x * f;
        return *this;
    }
    explicit operator bool() const {
        return state;
    }

private:
    bool state;
    static constexpr int maxn = 1 << 21;
    char buffer[maxn];
} qread;

```